

Java 23 — Primitive Types in Patterns, instanceof, and switch (Preview)

Aspect	Details
Java Version	Java 23
Year	2024
Feature	Primitive Types in Patterns, instanceof, and switch — Preview
UniqueClassName	PrimitiveTypesPatterns
JEP	JEP 455
Main purpose	Allows primitive types such as int, long, double, etc. to participate directly in pattern matching and switch, including guarded/range-style patterns.
Problem solved	Traditional pattern matching was primarily designed around reference types. Handling primitive values often required explicit comparisons, casts, or conversions.
Status	Preview in Java 23; preview features require compilation/runtime preview flags.

1. What does the feature provide?

The feature extends pattern matching so that primitive values can be used in patterns.

For example, conceptually:

```
static String classify(int value) {  
    return switch (value) {  
        case 0 -> "zero";  
        case int i when i > 0 -> "positive";  
        case int i -> "negative";  
    };  
}
```

It also allows patterns such as:

```
if (obj instanceof int i) {  
    System.out.println(i);  
}
```

The important idea is that **primitive values can participate directly in pattern matching rather than always being handled through ordinary comparisons or boxing.**

2. Advantages and disadvantages

Advantages	Explanation	Example / Benefit
Cleaner primitive handling	Reduces the need for repetitive primitive comparisons and conversions.	case int i when i > 100 -> ...
Better switch expressiveness	Primitive values can be incorporated into pattern-oriented switch logic.	Classifying numeric values by ranges.
Range-based logic becomes clearer	Guards can express conditions close to the value being matched.	case int i when i >= 18 -> "Adult"
Works naturally with pattern matching	Brings primitive values closer to the programming model already used for reference-type patterns.	Primitive and reference patterns can participate in the same style of logic.
Potentially less boxing	Code can operate directly on primitive values instead of converting them to wrapper objects solely for pattern matching.	Avoid unnecessary Integer objects.
Improved readability	Matching and extracting a value can happen in one construct.	case int i when i > 0 -> ...
Useful for numeric classification	Business rules involving ranges can become easier to organize.	Age, score, temperature, quantity classifications.
More consistent pattern-matching model	Extends Java's pattern-matching direction beyond reference types.	Fits with instanceof and switch pattern matching.

3. When to use and when not to use

Situation	Use Primitive Types in Patterns?	Why
-----------	----------------------------------	-----

Complex numeric classification	✓ Yes	Patterns can make multiple cases easier to organize.
Numeric ranges in switch	✓ Yes	Particularly useful when several value ranges must be handled.
Pattern matching involving primitives and references	✓ Yes	Provides a consistent matching model.
Processing heterogeneous values	✓ Yes	Useful when values can arrive in different representations/types.
Domain rules based on numeric categories	✓ Yes	Makes classification logic explicit.
Performance-sensitive primitive calculations	□ Evaluate	Use it for clarity, but benchmark rather than assuming performance benefits.
Simple $x > 100$ check	✗ Usually no	Ordinary comparison is simpler.
Simple equality check	✗ Usually no	$x == 10$ is already concise.
Public production API requiring broad Java compatibility	✗ Usually no	Preview features can create compatibility constraints.
Legacy application restricted to finalized Java features	✗ No	Preview functionality may not be acceptable.
Very simple switch over constants	□ Usually no	Ordinary switch cases may be clearer.
Code maintained by a team unfamiliar with preview features	□ Usually no	Training and maintenance costs may outweigh the benefit.

Disadvantages	Explanation	Impact
Preview feature	Java 23 requires preview support.	Not ideal for production code that must remain compatible with

		non-preview Java configurations.
Syntax may change	Preview features can change before becoming final.	Future migration may be necessary.
Can be overused	Simple comparisons can become unnecessarily complicated when expressed as patterns.	if (x > 10) may be clearer than a pattern-based construct.
Learning curve	Developers must understand patterns, guards, primitive/reference conversions, and switch semantics.	More concepts for teams to learn.
Migration considerations	Code written against the preview syntax may require changes in a future Java release.	Maintenance overhead.
Not automatically faster	The feature primarily improves expressiveness and type safety; it should not be assumed to provide a performance improvement.	Benchmark performance-critical code.
Boundary cases need care	Numeric conversions and widening/narrowing behavior can matter when different primitive types are involved.	Be careful with byte, short, int, long, float, and double.
May obscure simple business logic	Pattern matching is not always the most readable option.	Straightforward predicates may be preferable.

4. Real-time use cases

Real-time use case	Typical requirement	How the feature can help	Example
Age classification	Categorize ages	Match primitive int values and ranges	0–12, 13–17, 18+
Transaction amount	Identify transaction	Pattern/guard can	< ₹1,000, ₹1,000–

classification	tiers	express ranges	₹10,000, > ₹10,000
HTTP status classification	Categorize response codes	Match numeric values/ranges	2xx, 4xx, 5xx
Temperature monitoring	Categorize sensor readings	Match primitive numeric ranges	Cold / Normal / Hot
Exam score classification	Assign grades	Express numeric ranges	A / B / C / Fail
Inventory quantity	Identify stock levels	Match quantities against thresholds	Out of stock / Low / Normal
Network metrics	Categorize latency	Primitive pattern + guard	Fast / Acceptable / Slow
Payment processing	Categorize amounts	Numeric classification	Small / Medium / Large transaction
Device telemetry	Interpret sensor values	Pattern-oriented numeric processing	Normal / Warning / Critical
Game scoring	Classify player scores	Range-based matching	Beginner / Intermediate / Expert

5. Example — score classification

Traditional approach

```
static String grade(int score) {
    if (score >= 90) {
        return "A";
    } else if (score >= 75) {
        return "B";
    } else if (score >= 60) {
        return "C";
    } else {
        return "F";
    }
}
```

Pattern-oriented approach

```
static String grade(int score) {
    return switch (score) {
```

```

    case int s when s >= 90 -> "A";
    case int s when s >= 75 -> "B";
    case int s when s >= 60 -> "C";
    default -> "F";
  };
}

```

The second approach can make a larger set of numeric classification rules easier to organize, particularly when the surrounding logic already uses pattern matching.

6. Example — primitive and reference values

One motivation behind JEP 455 is making pattern matching more expressive when primitive and reference values are involved.

Conceptually:

Object value = 42;

```

if (value instanceof int i) {
    System.out.println("Integer value: " + i);
}

```

This style avoids requiring the developer to think primarily in terms of wrapper types when the desired result is a primitive value.

Important: because this was a preview feature in Java 23, exact syntax and semantics should be checked against the specific Java release being targeted.

7. Alternate features

Requirement	Alternative feature	When the alternative is better
Simple primitive comparison	if / else if	When there are only one or two conditions.
Fixed primitive values	Traditional switch	When matching exact constants rather than patterns/ranges.
Complex boolean conditions	if / else	When conditions are easier to read as predicates.
Object type matching	instanceof pattern matching	When working primarily with reference types.

Data-oriented decomposition	Record patterns	When extracting components from records.
Restricted type hierarchies	Sealed classes + pattern switch	When modeling a closed set of object types.
Numeric range classification	Ordinary comparisons	When the ranges are simple and pattern matching adds little value.
Functional transformation	Streams / lambdas	When processing collections rather than matching an individual value.
Maximum compatibility	Standard Java syntax	When preview features cannot be used in production.

8. Primitive Types in Patterns vs alternatives

Feature	Primitive Patterns	if/else	Traditional switch	instanceof Patterns	Record Patterns
Primitive values	★★★★★	★★★★★	★★★★	★★	★
Range classification	★★★★★	★★★★★	★★★	★	★
Exact-value matching	★★★★	★★★	★★★★★	★	★
Type matching	★★★★★	★★★	★★★	★★★★★	★★★★
Data decomposition	★★	★★	★★	★★★	★★★★★
Simplicity	★★★	★★★★★	★★★★★	★★★★	★★★
Preview dependency in Java 23	Yes	No	No	Depending on specific pattern feature	Depending on specific pattern feature
Best use	Numeric/type-oriented pattern logic	Simple conditions	Fixed alternatives	Object type checks	Decomposing structured data

9. Overall recommendation

Question	Recommendation
Should I learn it?	✓ Yes — especially if learning modern Java pattern matching.
Should I immediately use it everywhere?	✗ No .
Best reason to use it	Cleaner integration of primitive values with Java's pattern-matching model.
Best production scenario	After the feature becomes finalized and your project's Java version supports it as a standard feature.
Best alternative today for simple logic	if/else or ordinary switch.
Biggest advantage	More expressive primitive-oriented pattern matching.
Biggest disadvantage	It was a preview feature in Java 23 , so production adoption requires careful consideration of Java-version strategy.
Performance expectation	Treat it primarily as a language/readability feature , not a performance optimization.

Bottom line: Primitive Types in Patterns, instanceof, and switch`` is most valuable when your application has **non-trivial numeric/type classification logic** and you want that logic expressed using Java's pattern-matching model. For simple comparisons, ordinary if statements remain clearer.

Java 23 — Class-File API (Second Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Class-File API (Second Preview)
UniqueClassName	ClassFileAPI
JEP	JEP 466
Purpose	Provides a standard API for parsing, generating, and transforming

	Java class files (.class) without requiring developers to manipulate raw class-file bytes manually.
Introduced	Java 22 as a first preview
Java 23 status	Second Preview
Primary package	java.lang.classfile
Main users	Framework developers, bytecode tools, JVM tooling, instrumentation libraries, code generators

1. What is the Class-File API?

The Class-File API provides a modern Java API for working directly with JVM class files.

Traditionally, Java developers commonly used third-party bytecode libraries such as ASM, Byte Buddy, or Javassist to generate or modify .class files.

The Class-File API provides a **standard JDK API** for these operations.

Conceptually:

```
import java.lang.classfile.ClassFile;

byte[] classBytes = ...;

var classModel = ClassFile.of().parse(classBytes);
```

It can also be used to generate class files:

```
byte[] bytes = ClassFile.of().build(
    ClassDesc.of("com.example.Hello"),
    classBuilder -> {
        // Define class structure
    }
);
```

The exact API should be checked against the targeted Java release because this was still a preview API in Java 23.

2. Advantages

Advantage	Explanation	Practical benefit
Standard JDK API	Bytecode manipulation becomes part of the Java platform rather than relying exclusively on external libraries.	Fewer external dependencies.
Modern API design	Designed specifically around Java class-file structures.	More natural than manually manipulating byte arrays.
Class-file generation	Can programmatically create .class files.	Useful for code generators and runtime frameworks.
Class-file parsing	Can inspect existing class files.	Useful for analysis, tooling, and diagnostics.
Class-file transformation	Can modify class-file structures.	Useful for instrumentation and optimization tools.
Better maintainability	Structured API is easier to reason about than manually manipulating binary class-file data.	Lower maintenance burden for bytecode tooling.
JDK alignment	Designed by the Java platform team with JVM/class-file evolution in mind.	Easier long-term alignment with new class-file versions.
Potentially fewer dependencies	Applications/tools may not need a separate bytecode library for basic operations.	Smaller dependency footprint.
Useful for framework development	Frameworks can generate or inspect bytecode directly.	Enables advanced runtime capabilities.
Supports class-file evolution	The API is intended to evolve with JVM class-file changes.	Reduces some compatibility burden compared with custom parsers.
Good for tooling	Makes class-file information programmatically accessible.	Useful for IDEs, analyzers, profilers, and build tools.

Type-safe abstraction	Provides structured representations instead of treating a class file as an arbitrary byte array.	Fewer low-level byte-manipulation errors.
------------------------------	--	---

3. Disadvantages

Disadvantage	Explanation	Practical impact
Preview API in Java 23	It was not yet finalized.	API changes could require migration.
Advanced complexity	JVM class files contain many low-level concepts.	Significant learning curve.
Not needed by normal applications	Most business applications never need to manipulate bytecode.	Adding it can unnecessarily complicate a project.
Requires JVM knowledge	Developers need to understand classes, methods, descriptors, attributes, bytecode, etc.	Specialized expertise may be required.
Debugging can be difficult	Generated/modified bytecode can fail at class loading or verification time.	Errors can be harder to diagnose than normal Java errors.
Potential compatibility concerns	Generated class files must be valid for the target JVM/class-file version.	Deployment strategy matters.
Security implications	Dynamically generated classes can introduce security concerns if generated from untrusted input.	Input validation is important.
Poor fit for ordinary business logic	Bytecode manipulation is usually infrastructure-level work.	Don't use it just to avoid writing normal Java code.
Migration from existing libraries	Existing projects may already depend heavily on ASM/Byte Buddy/Javassist.	Replacing a mature library may provide little immediate value.

Preview feature management	Compilation and execution require preview-feature handling in Java 23.	Additional build/deployment configuration.
Generated-code complexity	Generated classes can be difficult for developers to inspect and maintain.	Testing and diagnostics become important.

4. When should you use it?

Scenario	Use?	Reason
Building a bytecode-generation framework	✓ Yes	This is one of the primary target scenarios.
Building JVM/class-file analysis tools	✓ Yes	Parsing .class files is directly supported.
Building an instrumentation framework	✓ Yes	Class transformation is a natural use case.
Creating classes dynamically	✓ Yes	Avoids manually constructing class-file binary structures.
Building compiler/tooling infrastructure	✓ Yes	Can inspect and generate class files.
Building a Java agent	✓ Potentially	Useful for class transformation.
Building an AOP framework	✓ Potentially	Can modify generated class behavior.
Building a code-generation framework	✓ Yes	Generated Java classes can be represented as class files.
Creating a simple REST API	✗ No	Normal Java classes are sufficient.
Building a CRUD application	✗ No	No meaningful need for bytecode manipulation.
Implementing ordinary	✗ No	Adds unnecessary complexity.

business logic		
Simple reflection requirement	✗ Usually no	Java Reflection is considerably simpler.
Simple object creation	✗ No	Constructors/factories are preferable.
Existing project already using mature bytecode tooling	☐ Evaluate	Migration may not provide enough benefit.

5. Real-time use cases

Real-time use case	What it does	How Class-File API helps
AOP framework	Adds logging, security, transactions, etc.	Modifies/generated class bytecode.
Java agent	Instruments running applications.	Transforms classes during class loading.
ORM framework	Generates optimized entity classes.	Generates or transforms bytecode.
Mocking framework	Creates runtime mock implementations.	Dynamically generates classes.
Proxy generation	Creates runtime proxy classes.	Generates class files programmatically.
Performance instrumentation	Adds timing/counters to methods.	Inserts bytecode around method execution.
Code coverage	Instruments classes to record execution.	Adds instrumentation to class files.
Static analysis tool	Inspects compiled classes.	Parses .class structures.
Dependency analysis	Determines class/method relationships.	Reads class-file metadata.
Build-time optimizer	Modifies generated class files.	Reads and transforms bytecode.
Plugin systems	Dynamically generate adapter classes.	Creates classes matching runtime requirements.

JVM educational tools	Shows how Java source becomes bytecode.	Reads class-file structures directly.
------------------------------	---	---------------------------------------

6. Example — analyzing a class file

Suppose we have:

```
public class CustomerService {

    public void saveCustomer() {
        System.out.println("Saving customer");
    }
}
```

After compilation:

```
CustomerService.java
    ↓
  javac
    ↓
CustomerService.class
```

A tool using the Class-File API can parse the generated class:

```
import java.lang.classfile.ClassFile;

byte[] classBytes = ...;

var classModel = ClassFile.of().parse(classBytes);

System.out.println(classModel);
```

This is useful for tools that need to inspect compiled classes without writing their own class-file parser.

7. Example — bytecode generation

A framework might dynamically create something conceptually equivalent to:

```
public class GeneratedCustomerService {

    public void execute() {
        System.out.println("Generated method");
    }
}
```

```
}  
}
```

Instead of creating a .java source file and invoking javac, a bytecode-generation framework can construct the class-file representation directly.

Conceptually:

```
Application metadata  
↓  
Class-File API  
↓  
Generated .class  
↓  
ClassLoader  
↓  
Runtime class
```

This is particularly useful for **frameworks and infrastructure**, rather than normal application code.

8. Example — runtime instrumentation

Suppose a framework wants to turn:

```
public void processOrder() {  
    orderRepository.save();  
}
```

into behavior conceptually equivalent to:

```
public void processOrder() {  
    long start = System.nanoTime();  
  
    try {  
        orderRepository.save();  
    } finally {  
        metrics.record(System.nanoTime() - start);  
    }  
}
```

A bytecode instrumentation framework can transform the compiled class so that the monitoring logic is inserted automatically.

The application developer does not have to manually add the timing code to every method.

This is useful for:

- observability
- performance monitoring
- tracing
- security instrumentation
- profiling
- code coverage

9. Class-File API vs Reflection

These two are often confused, but they solve very different problems.

Requirement	Class-File API	Reflection
Inspect loaded class	☐ Possible but not its primary purpose	✓ Excellent
Inspect .class file	✓ Excellent	✗ No
Generate .class file	✓ Yes	✗ No
Modify bytecode	✓ Yes	✗ No
Invoke a method dynamically	✗ Not its purpose	✓ Yes
Read annotations from loaded classes	☐	✓ Yes
Runtime framework metadata	☐	✓ Yes
Bytecode instrumentation	✓	✗
Class generation	✓	✗
Simplicity	☆☆	☆☆☆☆☆

Rule of thumb:

Use **Reflection** to inspect/use a class at runtime.

Use **Class-File API** to work with the **compiled class-file representation itself**.

10. Alternate features / technologies

Alternative	Type	Use it when	Compared with Class-File API
ASM	Bytecode manipulation library	You need mature, low-level JVM bytecode manipulation.	More established; external dependency.
Byte Buddy	High-level bytecode generation/manipulation	You want easier dynamic class generation/instrumentation.	Higher-level abstraction.
Javassist	Bytecode manipulation library	You want relatively simple runtime class modification.	Easier for some use cases, but different abstraction/model.
Java Reflection	JDK runtime API	You only need to inspect/invoke loaded classes.	Much simpler but cannot generate/transform arbitrary class files.
MethodHandle / MethodHandles	JDK runtime API	You need dynamic method invocation and linkage.	Runtime behavior rather than class-file generation.
Java Compiler API	JDK compilation API	You want to generate Java source and compile it.	Higher-level than generating class files directly.
java.lang.instrument	JDK instrumentation API	You need to transform classes as they are loaded.	Often used together with a bytecode manipulation API.
invokedynamic	JVM instruction mechanism	You need dynamic method linkage/call-site behavior.	Lower-level JVM mechanism; not a general class-file API.

11. Class-File API vs ASM vs Byte Buddy

Capability	Class-File API	ASM	Byte Buddy
Part of JDK	✓	✗	✗
External dependency	✗	✓	✓

Low-level bytecode control	★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★
High-level class generation	★ ★ ★	★ ★	★ ★ ★ ★ ★
Class-file parsing	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★
Instrumentation	★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Learning difficulty	High	High	Medium
JVM-oriented	✓	✓	✓
Mature ecosystem	Growing	Very mature	Very mature
Preview status in Java 23	☐ Yes	N/A	N/A
Best suited for	JDK-native class-file tooling	Low-level bytecode work	High-level runtime code generation

12. When NOT to replace ASM/Byte Buddy

If your application already uses a mature library such as ASM or Byte Buddy, **don't automatically replace it just because the JDK now provides the Class-File API.**

Consider:

Situation	Recommendation
Existing ASM-based framework is stable	Keep ASM unless there is a strong reason to migrate.
Existing Byte Buddy-based instrumentation	Usually keep Byte Buddy.
Need a high-level API	Byte Buddy may be more convenient.
Need very low-level bytecode control	ASM may still be preferable.
Want to minimize dependencies	Evaluate Class-File API.
Building a new JDK-focused tool	Class-File API is worth evaluating.

Need Java 23 preview functionality	Class-File API can be evaluated experimentally.
Need long-term stable production API	Prefer finalized APIs unless preview adoption is intentional.

13. Advantages vs disadvantages summary

Category	Assessment
Dependency reduction	☐ Excellent
Class-file parsing	☐ Excellent
Class generation	☐ Excellent
Instrumentation	☐ Very useful
Normal application development	☐ Usually unnecessary
Ease of learning	☐ Advanced
Runtime framework development	☐ Excellent
Preview stability in Java 23	☐ Limitation
Long-term JDK integration	☐ Strong potential
Replacement for every bytecode library	☐ No
Replacement for Reflection	☐ No
Performance optimization by itself	☐ Not inherently
Tool/framework development	☐ Excellent

Final recommendation

Question	Answer
Should I learn it?	✓ Yes , especially if you work on JVM internals, frameworks, agents, compilers, instrumentation, or bytecode tooling.
Should ordinary Java developers use it?	✗ Usually not.

Best use case	Parsing, generating, and transforming .class files.
Best alternative for high-level bytecode generation	Byte Buddy
Best alternative for low-level bytecode manipulation	ASM
Best alternative for runtime class inspection	Reflection
Biggest advantage	A standard JDK API for class-file manipulation.
Biggest disadvantage	It was still a preview API in Java 23 , and bytecode programming is inherently complex.
Performance benefit	Not automatically a performance feature; its main value is capability, standardization, and maintainability .
Production recommendation for Java 23	Use only if your project deliberately accepts preview APIs; otherwise wait for a finalized release or use a mature alternative.

Java 23 — Markdown Documentation Comments

Aspect	Details
Java Version	23
Year	2024
Feature	Markdown Documentation Comments
UniqueClassName	MarkdownDocumentationComments
JEP	467 — Markdown Documentation Comments
Status in Java 23	Final
Purpose	Allows Java API documentation comments to be written using Markdown , rather than relying primarily on traditional Javadoc

	syntax.
Main benefit	Makes documentation easier to write and read in source code while allowing javadoc to produce HTML documentation.
Primary tool	Javadoc

1. What are Markdown Documentation Comments?

Before Java 23, Javadoc traditionally used HTML inside documentation comments:

```
/**
 * Calculates the total.
 *
 * <p>Returns the sum of {@code price} and {@code tax}.
 *
 * @param price product price
 * @param tax tax amount
 * @return total amount
 */
public double calculateTotal(double price, double tax) {
    return price + tax;
}
```

Java 23 allows Markdown syntax inside documentation comments:

```
/// Calculates the total.
///
/// Returns the sum of `price` and `tax`.
///
/// - `price` is the product price.
/// - `tax` is the tax amount.
///
/// @param price product price
/// @param tax tax amount
/// @return total amount
public double calculateTotal(double price, double tax) {
    return price + tax;
}
```

You can use Markdown constructs such as:

- headings
- lists
- emphasis
- code spans

- fenced code blocks
- links
- tables
- block quotes

The important point is that **Javadoc remains the documentation-generation mechanism**; Markdown changes how developers author the documentation.

2. Advantages

Advantage	Explanation	Practical benefit
More readable source code	Markdown is generally easier to read directly in a Java source file than HTML-heavy Javadoc.	Developers understand documentation faster.
Less HTML noise	You don't need to write tags such as <code><p></code> , <code></code> , <code></code> , and <code><code></code> for common documentation.	Cleaner source code.
Natural code formatting	Inline code can use backticks.	<code>calculateTotal()</code> is easier to write.
Lists are simpler	Markdown lists replace verbose HTML list markup.	Easier API documentation.
Headings are easier	Markdown headings can organize long documentation.	Better structure for large APIs.
Fenced code blocks	Code examples can be presented naturally.	Excellent for API usage examples.
Better developer experience	Markdown is widely known by modern developers.	Lower documentation learning curve.
Documentation becomes easier to maintain	Less verbose markup means fewer formatting mistakes.	Lower maintenance cost.
Good for examples	Multi-line code examples are easier to present.	Better API documentation.
Works with Javadoc	Existing Javadoc	Existing Java documentation tooling

	generation remains part of the workflow.	remains relevant.
Supports richer documentation	Markdown makes longer API documentation more approachable.	Useful for frameworks and libraries.
Consistent with modern documentation practices	Developers commonly use Markdown for README files and technical documentation.	Easier transition between source documentation and project documentation.

3. Disadvantages

Disadvantage	Explanation	Impact
Java 23+ feature	Projects targeting older Java versions cannot assume this syntax is available.	Version compatibility matters.
Javadoc knowledge is still required	Markdown does not replace Javadoc concepts such as @param, @return, @throws, etc.	Developers still need Javadoc knowledge.
Not every Markdown feature is necessarily appropriate	Javadoc processes Markdown for documentation purposes; it isn't a general-purpose Markdown renderer.	Don't assume every Markdown extension works.
Mixed syntax can be confusing	Existing projects may contain traditional Javadoc, HTML, and Markdown together.	Documentation style guidelines become important.
Migration may take time	Large projects with existing HTML-heavy Javadocs may require gradual conversion.	Migration effort.
Tooling compatibility	Third-party documentation processors may not support all Java 23 Markdown behavior immediately.	Verify your documentation toolchain.

Markdown doesn't improve runtime performance	This is a documentation feature.	No application-performance benefit.
Can encourage overly long comments	Markdown makes rich documentation easier, but excessive documentation can become difficult to maintain.	Keep API docs focused.
Still produces generated documentation	Developers still need to run Javadoc or an equivalent documentation pipeline.	It doesn't eliminate the documentation build process.

4. When to use

Situation	Use?	Reason
Public Java library/API	✓ Yes	Clear API documentation is extremely valuable.
Enterprise shared libraries	✓ Yes	Helps developers understand reusable components.
Framework development	✓ Yes	Framework APIs often require extensive explanations and examples.
Complex business APIs	✓ Yes	Markdown improves readability of detailed documentation.
Methods with complicated behavior	✓ Yes	Examples, lists, and explanations are useful.
Configuration APIs	✓ Yes	Markdown can explain configuration options clearly.
SDK development	✓ Yes	Consumer-facing API documentation benefits significantly.
Utility methods with non-obvious behavior	✓ Yes	Documents assumptions and edge cases.
Internal implementation details	□ Sometimes	Document only when future maintainers genuinely need the

		information.
Very simple private getter	✗ Usually no	The method name may already explain enough.
Trivial setter	✗ Usually no	Avoid documentation noise.
Self-explanatory code	✗ Usually no	Don't document what the code already makes obvious.
Runtime business logic	✗ Not for performance	Documentation has no runtime performance advantage.
Project using Java <23	✗	Use the documentation syntax supported by your target Java version/toolchain.

5. Real-time use cases

Real-time use case	Example	Why Markdown helps
Public REST SDK	Documenting <code>createCustomer()</code>	Lists parameters, errors, and examples cleanly.
Payment library	Documenting payment APIs	Can explain supported payment states and exceptions.
Spring-based enterprise library	Shared service APIs	Easier internal developer documentation.
Database framework	Query/transaction APIs	Code examples become more readable.
Security library	Authentication APIs	Can clearly describe security requirements and warnings.
Cloud SDK	Service client methods	Useful for request/response examples.
Java utility library	Date/string/collection utilities	Provides concise examples and edge cases.
Developer platform	Plugin APIs	Markdown makes integration instructions easier to read.
Open-source project	Public classes and	Familiar documentation syntax for

	interfaces	contributors.
Large enterprise application	Shared domain APIs	Makes internal APIs easier to discover and understand.

6. Example — traditional Javadoc vs Markdown

Traditional Javadoc

```
/**
 * Processes an order.
 *
 * <p>The order must have a valid customer and at least one item.</p>
 *
 * <ul>
 *   <li>Validates the order.</li>
 *   <li>Charges the customer.</li>
 *   <li>Updates inventory.</li>
 * </ul>
 *
 * @param order order to process
 * @return processing result
 * @throws IllegalArgumentException if the order is invalid
 */
public OrderResult process(Order order) {
    // ...
}
```

Java 23 Markdown style

```
/**
 * Processes an order.
 *
 * The order must have a valid customer and at least one item.
 *
 * Processing performs:
 *
 * - Order validation
 * - Customer charging
 * - Inventory update
 *
 * @param order order to process
 * @return processing result
 * @throws IllegalArgumentException if the order is invalid
 */
public OrderResult process(Order order) {
```

```
} // ...
```

The second version is generally easier to read **inside the Java source file**.

7. Example — code documentation

Markdown is particularly useful when an API needs a usage example.

```
/**
 * Creates a customer.
 *
 * Example:
 *
 * ```java
 * Customer customer = client.createCustomer(
 *     new CustomerRequest("Anurag", "anurag@example.com")
 * );
 * ```
 *
 * The returned customer contains the generated customer ID.
 *
 * @param request customer information
 * @return newly created customer
 */
public Customer createCustomer(CustomerRequest request) {
    // ...
}
```

This style is particularly valuable for **SDKs, libraries, and frameworks**, where documentation should answer not only *what* a method does but also *how* developers should use it.

8. Example — API documentation with a table

Markdown is useful for structured information.

```
/**
 * Supported customer states:
 *
 * | State | Meaning |
 * |---|---|
 * | `ACTIVE` | Customer can place orders |
 * | `SUSPENDED` | Customer cannot place orders |
 * | `CLOSED` | Customer account is permanently closed |
 */
```

```

* @param customer customer to evaluate
* @return whether the customer can place orders
*/
public boolean canPlaceOrder(Customer customer) {
    // ...
}

```

This can be much more readable than manually constructing an HTML table.

9. Markdown Documentation Comments vs traditional Javadoc

Capability	Markdown Documentation Comments	Traditional Javadoc
Readable in source	★ ★ ★ ★ ★	★ ★ ★
HTML verbosity	★ ★ ★ ★ ★ Low	★ ★ High
@param	✓	✓
@return	✓	✓
@throws	✓	✓
Lists	★ ★ ★ ★ ★	★ ★ ★
Code examples	★ ★ ★ ★ ★	★ ★ ★
Tables	★ ★ ★ ★ ★	★ ★
Familiar to modern developers	★ ★ ★ ★ ★	★ ★ ★
Existing Java ecosystem	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Best for new documentation	★ ★ ★ ★ ★	★ ★ ★
Best for legacy Java projects	★ ★ ★	★ ★ ★ ★ ★
Requires Java 23 Markdown capability	✓	✗
Runtime performance impact	None	None

10. Alternate features

Requirement	Alternative	When to choose it
Standard Java API documentation	Traditional Javadoc	When targeting older Java versions or maintaining existing projects.
Project-level documentation	Markdown README/docs	When documenting the entire project rather than a specific API element.
Rich documentation website	Javadoc + documentation site generator	For large public libraries/frameworks.
Architecture documentation	Markdown + diagrams	For system-level documentation.
API specification	OpenAPI	For HTTP/REST APIs rather than Java method documentation.
Developer guides	Markdown documentation	For tutorials and integration guides.
IDE API information	Javadoc	When documentation should appear through IDE hover/autocomplete.
Code contracts	Javadoc annotations/tags	When documenting parameters, return values, exceptions, and API contracts.

11. Markdown Documentation Comments vs README Markdown

Aspect	Markdown Javadoc	README Markdown
Scope	Class/method/field/package	Whole project
Generated by javadoc	✓	✗ Usually
Appears with API documentation	✓	✗

API method documentation	★ ★ ★ ★ ★	★ ★
Installation instructions	★ ★	★ ★ ★ ★ ★
Tutorial	★ ★ ★	★ ★ ★ ★ ★
Architecture overview	★ ★ ★	★ ★ ★ ★ ★
Method parameters	★ ★ ★ ★ ★	★ ★
IDE integration	★ ★ ★ ★ ★	★
Best use	API reference	Project/developer guide

12. Recommended documentation strategy

For a modern Java project, don't treat Markdown Javadoc as a replacement for **all** documentation.

A good structure is:

```

Project
├── README.md
│   └── Installation / Quick start
├── Architecture documentation
│   └── Design / decisions / diagrams
├── Java source
│   └── Markdown Javadoc
│       ├── API behavior
│       ├── Parameters
│       ├── Return values
│       ├── Exceptions
│       └── Usage examples

```

13. Advantages vs disadvantages — final assessment

Category	Assessment
Source-code readability	☐ Excellent
Documentation maintainability	☐ Excellent
API documentation	☐ Excellent

Code examples	☐ Excellent
Tables/lists	☐ Excellent
Learning curve	☐ Low for developers familiar with Markdown
Runtime performance	☐ No meaningful effect
Legacy Java compatibility	☐ Depends on target/toolchain
Large public APIs	☐ Highly recommended
Simple private methods	☐ Usually unnecessary
Replacement for README	☐ No
Replacement for all Javadoc concepts	☐ No
Replacement for OpenAPI	☐ No
Best alternative	Traditional Javadoc for older/legacy Java projects

Final recommendation

Question	Answer
Should I use it for new Java 23+ libraries?	✓ Yes
Should every method have a Markdown comment?	✗ No
Where is it most valuable?	Public APIs, SDKs, frameworks, shared libraries, and complex APIs.
What should still be used?	@param, @return, @throws, and other Javadoc tags where they communicate API contracts.
Is it a performance feature?	✗ No. It affects documentation authoring , not application execution.
Best alternative for Java 8–22 projects?	Traditional Javadoc.

Best complementary technology?	Project-level Markdown documentation/README plus generated Javadoc.
Biggest advantage	Documentation becomes much easier to read and write directly in Java source.
Biggest disadvantage	Projects must consider Java/toolchain compatibility and the interaction between Markdown and existing Javadoc/HTML documentation.

Java 23 — Vector API (Eighth Incubator)

Aspect	Details
Java Version	23
Year	2024
Feature	Vector API (Eighth Incubator)
UniqueClassName	VectorAPI
JEP	469 — Vector API (Eighth Incubator)
Status	Incubator
Main package	jdk.incubator.vector
Primary goal	Enable Java applications to express vector/SIMD computations so the JIT compiler can compile them into efficient CPU vector instructions.
Main target	CPU-intensive numerical and data-parallel workloads
Typical operations	Addition, multiplication, comparison, masking, reductions, rearrangement, etc.
Key idea	Process multiple primitive values in parallel instead of processing one value at a time.

1. What is the Vector API?

Normally, code such as:

```
for (int i = 0; i < size; i++) {  
    result[i] = a[i] + b[i];  
}
```

conceptually processes:

```
a[0] + b[0]  
a[1] + b[1]  
a[2] + b[2]  
a[3] + b[3]  
...
```

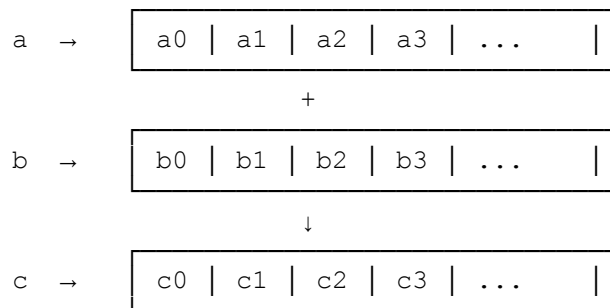
The Vector API allows the computation to be expressed as a vector operation:

```
var va = IntVector.fromArray(species, a, i);  
var vb = IntVector.fromArray(species, b, i);
```

```
var result = va.add(vb);
```

```
result.intoArray(c, i);
```

Conceptually, the CPU can perform:



This is known as **SIMD — Single Instruction, Multiple Data**.

2. Advantages

Advantage	Explanation	Practical benefit
SIMD processing	Multiple primitive values can be processed by one vector	Potentially much higher throughput.

	instruction.	
Excellent for data-parallel algorithms	The same operation is performed independently on many values.	Ideal for arrays, matrices, images, signals, etc.
Can exploit CPU vector instructions	The JVM JIT can map vector operations to available hardware instructions.	Can take advantage of SIMD capabilities without writing architecture-specific assembly.
Portable Java API	Java code can express vector computations without directly writing x86/ARM intrinsics.	Better portability than CPU-specific native code.
Works with different CPU architectures	Designed to map to supported vector hardware on different platforms.	Useful for x86 and AArch64 environments.
Potentially significant performance improvement	CPU-intensive loops may process several elements simultaneously.	Faster numerical workloads.
Explicit vectorization	Developers can express vector operations when ordinary auto-vectorization isn't sufficient.	Greater control over performance-critical code.
Supports masks	Operations can selectively apply to vector lanes.	Useful for conditional calculations.
Supports reductions	Multiple vector elements can be reduced to one result.	Useful for sums, minimums, maximums, etc.
Supports rearrangement	Vector elements can be rearranged.	Useful in image processing, algorithms, and data transformations.
Avoids JNI for many SIMD tasks	Pure Java can express vector operations.	Less native-code integration overhead.
Useful for high-throughput systems	Especially beneficial when millions/billions of calculations are performed.	Can improve throughput significantly.

3. Disadvantages

Disadvantage	Explanation	Practical impact
Incubator API	It was still an incubating API in Java 23.	API can change in future releases.
Not for ordinary application code	Most applications don't need explicit SIMD programming.	Adds unnecessary complexity.
Requires performance expertise	Developers need to understand data locality, vector width, CPU behavior, JIT compilation, and benchmarking.	Steeper learning curve.
Performance is workload-dependent	Vectorization does not guarantee a particular speedup.	Benchmark before adopting.
Memory bandwidth can become the bottleneck	Faster computation doesn't help if data cannot be supplied to the CPU quickly enough.	Some workloads see little improvement.
Branch-heavy algorithms are difficult	SIMD works best when operations are uniform across elements.	Complex branching can reduce benefits.
Small workloads may not benefit	Vector setup and data movement may outweigh the computational savings.	Scalar Java may be faster or simpler.
Hardware differences matter	Different CPUs have different vector capabilities.	Performance can vary across machines.
More complex code	Vector code is less straightforward than a normal Java loop.	Maintenance burden.
Debugging can be harder	Vector lane/mask behavior adds another dimension to debugging.	More testing may be required.
Requires incubator module handling	Applications need access to <code>jdk.incubator.vector</code> .	Build/deployment configuration is more involved.
Not automatically faster than JIT	Modern HotSpot can already auto-vectorize some ordinary loops.	Explicit Vector API isn't always necessary.

4. When to use

Situation	Use Vector API?	Reason
Large numerical arrays	✓ Yes	Excellent data-parallel workload.
Matrix/vector calculations	✓ Yes	Many independent arithmetic operations.
Image processing	✓ Yes	Pixels often undergo identical operations.
Audio/signal processing	✓ Yes	Samples can be processed in batches.
Machine-learning numerical kernels	✓ Yes	Large amounts of arithmetic on primitive data.
Financial numerical calculations	✓ Potentially	Useful for large independent calculations.
Scientific computing	✓ Yes	High numerical throughput can matter significantly.
Cryptographic algorithms	□ Potentially	Only where the algorithm maps well to vector operations and security requirements are satisfied.
Compression/decompression	✓ Potentially	Some algorithms have highly parallel operations.
Large-scale data transformation	✓ Potentially	Good when identical operations apply to many primitive values.
Simple CRUD application	✗ No	Little benefit.
Ordinary web controller	✗ No	Request handling isn't generally SIMD-bound.
Database CRUD	✗ No	Database/network latency dominates.
Small arrays	✗ Usually no	Overhead and complexity may outweigh benefits.

Branch-heavy processing	✗ Usually no	Poor SIMD characteristics.
I/O-bound application	✗ No	SIMD doesn't improve network/disk latency.
Code that is already fast enough	✗ No	Complexity isn't justified without measurable benefit.

5. Real-time use cases

Use case	Example workload	Why Vector API can help
Image processing	Brightness adjustment for millions of pixels	Same arithmetic operation can be applied to many pixels simultaneously.
Video processing	Pixel/color transformations	Large data-parallel workloads.
Audio processing	Volume adjustment/filtering	Many audio samples can be processed together.
Machine learning	Vector/matrix operations	Large amounts of numerical computation.
Scientific simulation	Physics calculations	Same equations applied to many data points.
Financial analytics	Risk calculations over large portfolios	Independent numerical calculations can be vectorized.
Signal processing	FFT/filtering-related kernels	Many numerical operations are data-parallel.
Computer vision	Pixel and feature calculations	Large arrays of numeric data.
Compression	Parallel byte/int transformations	Certain algorithms contain SIMD-friendly stages.
Search/index processing	Numeric filtering	Multiple values can be compared simultaneously.
Data analytics	Columnar numeric operations	Same operation across many primitive values.
Game engines	Physics/vector calculations	Large collections of independent numerical calculations.

--	--	--

6. Example — vector addition

Traditional scalar implementation

```
static void add(int[] a, int[] b, int[] c) {  
    for (int i = 0; i < a.length; i++) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Each iteration processes one pair:

```
a0 + b0 → c0  
a1 + b1 → c1  
a2 + b2 → c2  
a3 + b3 → c3
```

Vector API implementation

```
import jdk.incubator.vector.IntVector;  
import jdk.incubator.vector.VectorSpecies;  
  
static final VectorSpecies<Integer> SPECIES =  
    IntVector.SPECIES_PREFERRED;  
  
static void add(int[] a, int[] b, int[] c) {  
  
    int i = 0;  
    int upperBound = SPECIES.loopBound(a.length);  
  
    for (; i < upperBound; i += SPECIES.length()) {  
  
        var va = IntVector.fromArray(SPECIES, a, i);  
        var vb = IntVector.fromArray(SPECIES, b, i);  
  
        var vc = va.add(vb);  
  
        vc.intoArray(c, i);  
    }  
  
    // Handle remaining elements  
    for (; i < a.length; i++) {  
        c[i] = a[i] + b[i];  
    }  
}
```

The important difference is:

Scalar:

[a0] + [b0]

[a1] + [b1]

[a2] + [b2]

[a3] + [b3]

Vector:

[a0 a1 a2 a3] + [b0 b1 b2 b3]

↓

[c0 c1 c2 c3]

The actual number of lanes depends on the selected vector species and the underlying hardware.

7. Example — vector reduction

Suppose you need to calculate the sum of a very large array.

Traditional:

```
long sum = 0;
```

```
for (int value : values) {  
    sum += value;  
}
```

A Vector API implementation can process several values at once and then reduce them:

```
var sumVector = LongVector.zero(LongVector.SPECIES_PREFERRED);
```

```
for (int i = 0;  
    i < LongVector.SPECIES_PREFERRED.loopBound(values.length);  
    i += LongVector.SPECIES_PREFERRED.length()) {
```

```
    var vector = LongVector.fromArray(  
        LongVector.SPECIES_PREFERRED,  
        values,  
        i  
    );
```

```
    sumVector = sumVector.add(vector);  
}
```

```
long sum = sumVector.reduceLanes(VectorOperators.ADD);
```

This is particularly interesting for **large numerical datasets**.

8. Example — image processing

Imagine converting image pixels to grayscale.

A scalar implementation might perform:

```
for (int i = 0; i < pixels.length; i++) {  
    pixels[i] = convertToGray(pixels[i]);  
}
```

For millions of pixels, the same operation is repeated many times.

A SIMD-friendly implementation can process multiple pixel values together.

Pixels:

```
[P1 P2 P3 P4 P5 P6 P7 P8]
```

↓ Vector operation

```
[G1 G2 G3 G4 G5 G6 G7 G8]
```

This is a classic case where Vector API can potentially provide substantial throughput improvements.

9. Vector API vs ordinary Java loop

Factor	Ordinary loop	Vector API
Code simplicity	★ ★ ★ ★ ★	★ ★ ★
Portability	★ ★ ★ ★ ★	★ ★ ★ ★
Explicit SIMD control	★ ★	★ ★ ★ ★ ★
Potential numerical throughput	★ ★ ★	★ ★ ★ ★ ★
Learning difficulty	★ ★ ★ ★ ★	★ ★
Maintenance	★ ★ ★ ★ ★	★ ★ ★
Works for small workloads	★ ★ ★ ★ ★	★ ★ ★

Works for large data-parallel workloads	★ ★ ★ ★	★ ★ ★ ★ ★
CPU-specific optimization	★ ★ ★	★ ★ ★ ★ ★
Need benchmarking	Recommended	Essential

10. Vector API vs auto-vectorization

This is an important distinction.

Modern HotSpot can sometimes transform ordinary loops into SIMD instructions automatically:

```
for (int i = 0; i < length; i++) {
    result[i] = a[i] + b[i];
}
```

The JIT may recognize this as vectorizable.

Therefore:

Don't assume that replacing every loop with the Vector API will make your application faster.

Use the Vector API when you need **explicit control** or when benchmarking shows that it provides a meaningful advantage.

Scenario	Preferred approach
Simple numeric loop	Start with ordinary Java
JIT already vectorizes it effectively	Keep ordinary Java
JIT fails to vectorize an important hot loop	Consider Vector API
Complex SIMD operation	Consider Vector API
Performance-critical numerical kernel	Benchmark Vector API
Non-performance-critical code	Prefer simpler Java

11. Alternate features / technologies

Alternative	What it provides	When preferable
Ordinary Java loops	Scalar computation, potentially JIT-optimized	Default choice for most applications.
HotSpot auto-vectorization	JIT automatically generates SIMD instructions for suitable loops	When a normal loop already gets good performance.
Parallel Streams	Parallel processing across threads	When the workload benefits more from multi-core parallelism than SIMD.
Fork/Join Framework	Parallel task execution	For large independent tasks rather than lane-level SIMD operations.
GPU computing	Massive parallel numerical computation	Very large workloads suitable for GPUs.
JNI + native SIMD	Direct use of native SIMD libraries/instructions	When maximum hardware-specific control is required.
BLAS/native numerical libraries	Highly optimized mathematical operations	Matrix/scientific workloads where mature native libraries are appropriate.
Apache Commons Math	High-level mathematical algorithms	When productivity is more important than hand-optimized SIMD.
Panama/FFM	Native interoperability	When the required optimized implementation already exists in a native library.

12. Vector API vs Parallel Streams

These technologies solve different levels of parallelism.

Aspect	Vector API	Parallel Streams
Parallelism level	SIMD lanes	CPU threads
Typical unit	Vector elements	Collection elements/tasks

Main goal	CPU instruction-level parallelism	Multi-core parallelism
Thread creation	No application-level thread parallelism required	Uses ForkJoinPool/common pool
Best for	Numerical/data-parallel kernels	Independent collection operations
Example	$a[i] + b[i]$ across many values	Processing many independent objects
Overhead	Low once vectorized	Higher
Requires SIMD-friendly data	Yes	No
Can combine	✓ Yes	✓ Yes
Typical use	High-performance numeric code	General parallel data processing

13. Vector API vs JNI/native SIMD

Aspect	Vector API	JNI/native SIMD
Language	Java	C/C++/native
JVM integration	★ ★ ★ ★ ★	★ ★ ★
Portability	★ ★ ★ ★	★ ★
Hardware-specific control	★ ★ ★ ★	★ ★ ★ ★ ★
Development complexity	Medium/High	High
Native dependency	✗	✓
Memory boundary concerns	Lower	Potentially significant
Maintenance	Easier	More complex
Best for	Portable JVM SIMD	Maximum low-level control

14. When NOT to use Vector API

Don't use it when...	Reason
Your application is primarily CRUD	Database operations dominate performance.
Your application is network-bound	Network latency dominates.
Your workload is I/O-bound	SIMD won't improve disk/network operations.
Arrays are tiny	SIMD overhead may not be worthwhile.
Algorithm has significant branching	SIMD efficiency may be poor.
Data is irregular	Vectorization becomes difficult.
Ordinary Java already performs sufficiently well	Added complexity isn't justified.
You haven't profiled the application	You may optimize the wrong code.
Maintainability is more important than micro-optimization	Scalar Java is easier to understand.
You need a stable API and cannot accept incubator features	Java 23 Vector API was still incubating.

15. Recommended performance workflow

For performance-sensitive Java code, use this sequence:

1. Write clean ordinary Java
- ↓
2. Profile the application
- ↓
3. Identify CPU-hot methods
- ↓
4. Benchmark representative workloads
- ↓
5. Check whether HotSpot auto-vectorizes
- ↓
6. Try Vector API if justified
- ↓
7. Benchmark again
- ↓

8. Keep it only if the real workload improves

This is much safer than introducing the Vector API throughout an application simply because SIMD sounds faster.

16. Final comparison

Category	Vector API	Ordinary Java	Parallel Streams	Native SIMD
Ease of use	★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★
SIMD capability	★ ★ ★ ★ ★	★ ★ ★ *	★ ★	★ ★ ★ ★ ★
Multi-core parallelism	★ ★	★ ★	★ ★ ★ ★ ★	★ ★ ★ ★
Portability	★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★
Performance control	★ ★ ★ ★ ★	★ ★ ★	★ ★ ★	★ ★ ★ ★ ★
Dependency requirements	JDK incubator module	None	None	Native libraries
Maintenance	★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★
Best for numerical kernels	★ ★ ★ ★ ★	★ ★ ★	★ ★ ★	★ ★ ★ ★ ★
Best for CRUD/web apps	★	★ ★ ★ ★ ★	★ ★	★
Java 23 status	Incubator	Stable	Stable	External/native
Main risk	Complexity/API evolution	Missed optimization	Parallel overhead	Native complexity

* Ordinary Java loops may be automatically vectorized by the JIT, so their actual performance can be surprisingly good.

Final recommendation

Question	Answer
----------	--------

Should I learn it?	✓ Yes , if you're interested in high-performance Java, numerical computing, scientific applications, ML, image/video processing, or JVM internals.
Should every Java developer use it?	✗ No.
Best use case	Large, CPU-intensive, data-parallel numerical workloads.
First choice for ordinary applications	Regular Java code.
Before using Vector API	Profile and benchmark.
Biggest advantage	Explicit access to SIMD/vector computation from Java.
Biggest disadvantage	More complex code and the fact that it was still an incubator API in Java 23 .
Best alternative for simple workloads	Ordinary Java loops + HotSpot JIT.
Best alternative for thread-level parallelism	Parallel Streams or Fork/Join.
Best alternative for maximum native hardware control	Native SIMD libraries/code.
Performance guarantee?	✗ No. The actual improvement depends heavily on the algorithm, data size, memory behavior, CPU, and JIT/hardware capabilities.
Production strategy	Use it only in measured CPU hotspots where the benefit justifies the additional complexity and incubator dependency.

Java 23 — Stream Gatherers (Second Preview)

Aspect	Details
Java Version	23
Year	2024

Feature	Stream Gatherers (Second Preview)
UniqueClassName	StreamGatherers
JEP	473 — Stream Gatherers (Second Preview)
Status	Second Preview
Package	java.util.stream
Main API	Gatherer, Gatherers
Purpose	Provides a mechanism for defining custom intermediate stream operations that cannot be expressed conveniently using the standard Stream API.
Introduced	Java 22 as a first preview
Java 23 improvement	Refined and enhanced the Stream Gatherers API based on feedback from the first preview.

1. What are Stream Gatherers?

The traditional Stream API provides operations such as:

```
filter()
map()
flatMap()
distinct()
sorted()
limit()
skip()
takeWhile()
dropWhile()
```

However, some operations are difficult or awkward to express using these standard operations.

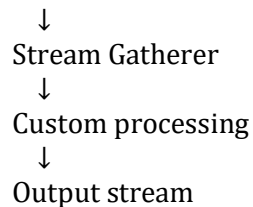
For example:

"Process elements in groups of 3."

A gatherer can express this type of custom intermediate operation.

Conceptually:

Input stream



For example:

```
var result = Stream.of(1, 2, 3, 4, 5, 6)
    .gather(Gatherers.windowFixed(3))
    .toList();
```

Conceptually:

Input:
1 2 3 4 5 6

↓
Gatherer

↓
[1 2 3]
[4 5 6]

This is significantly more expressive than trying to construct such behavior using multiple map, filter, and state-management operations.

2. Why Stream Gatherers were introduced

Traditional Stream limitation	Stream Gatherer solution
Difficult to implement custom stateful intermediate operations	Provides a structured mechanism for stateful operations
Custom windowing is awkward	windowFixed, windowSliding
Need to scan adjacent elements	fold, scan-style gatherers
Need custom batching	Gatherer can maintain a buffer
Need custom early termination	Gatherer supports controlled downstream processing
Need state across stream elements	Gatherer explicitly supports state

Need reusable custom stream operation	Implement a reusable Gatherer

3. Advantages

Advantage	Explanation	Practical benefit
Custom intermediate operations	Developers can create stream operations beyond the standard API.	Solves problems that previously required complicated pipelines.
Stateful processing	A gatherer can maintain state while processing elements.	Useful for windows, running calculations, batching, etc.
Supports windowing	Elements can be grouped into fixed or sliding windows.	Useful for analytics and time-series processing.
Can short-circuit	Gatherers can stop processing when appropriate.	Efficient for certain streaming algorithms.
Reusable abstraction	A custom gatherer can be reused across many pipelines.	Reduces duplicated stream logic.
Composes with Stream API	Gatherers fit into existing stream pipelines.	Can combine with filter, map, sorted, etc.
Cleaner than manual state management	Avoids external mutable variables in many cases.	Better functional-style code.
Can support parallel processing	The Gatherer abstraction includes mechanisms for parallel stream processing.	Potential scalability for suitable gatherers.
Better domain-specific operations	Teams can create operations specific to their domain.	Makes complex pipelines easier to express.
Can handle complex transformations	More flexible than simple one-element-at-a-time map.	Useful for algorithms requiring context from neighboring elements.
Improves Stream API expressiveness	Bridges the gap between basic stream operations and custom algorithms.	Fewer cases where Streams must be abandoned for manual loops.

4. Disadvantages

Disadvantage	Explanation	Practical impact
Preview feature in Java 23	It was still a second-preview feature.	API could change before finalization.
More difficult than standard Stream operations	Developers need to understand Gatherer, state, integrators, and downstream behavior.	Higher learning curve.
Can be over-engineered	A simple loop can sometimes be clearer.	Don't use gatherers merely because they exist.
State management can be complex	Stateful processing introduces additional reasoning requirements.	Bugs can be harder to diagnose.
Parallel behavior requires care	Custom gatherers need correct combiner/parallel semantics where applicable.	Incorrect implementations can produce unexpected results.
Debugging stream pipelines is already difficult	Gatherers add another abstraction layer.	Stack traces and data flow can be harder to follow.
Performance is workload-dependent	A gatherer isn't automatically faster than a loop.	Benchmark important workloads.
Not useful for simple transformations	map() and filter() are simpler for common cases.	Avoid unnecessary complexity.
Team familiarity	Many developers are already familiar with standard Streams but not Gatherers.	Training may be required.
Potential migration cost	Preview syntax/API can change.	Future Java upgrades may require modifications.

5. When to use Stream Gatherers

Situation	Use?	Reason
Custom stateful intermediate operation	✓ Yes	This is the primary purpose.

Fixed-size grouping/windowing	✓ Yes	Gatherers provide direct support.
Sliding windows	✓ Yes	Much cleaner than manually managing indexes/state.
Running calculations	✓ Yes	State can be maintained across elements.
Processing neighboring elements	✓ Yes	Standard map() is often insufficient.
Custom batching	✓ Yes	Gatherers can accumulate elements before producing output.
Custom short-circuiting operation	✓ Yes	Gatherers can control downstream processing.
Reusable complex stream operation	✓ Yes	Encapsulate the operation in a gatherer.
Simple filtering	✗ No	Use filter().
Simple transformation	✗ No	Use map().
Flattening collections	✗ No	Use flatMap().
Sorting	✗ No	Use sorted().
Removing duplicates	✗ No	Use distinct().
Simple aggregation	✗ Usually no	Use reduce(), collect(), or Collectors.
Very complex algorithm with imperative control flow	□ Evaluate	A normal loop may be clearer.

6. Real-time use cases

Use case	Requirement	Gatherer benefit
Batch processing	Process records in batches of 100	Gatherer can collect elements into batches.

Sliding-window analytics	Calculate metrics over the last N events	Sliding-window gatherer fits naturally.
Time-series processing	Analyze neighboring measurements	Maintains state between elements.
Log processing	Group related log events	Custom stateful grouping.
Fraud detection	Analyze sequences of transactions	Can maintain context across transactions.
Monitoring	Calculate rolling metrics	Stateful processing.
Event processing	Process event sequences	Access to prior/current elements.
Sensor data	Calculate moving averages	Sliding windows.
Financial analytics	Rolling averages/indicators	Window-based processing.
Recommendation systems	Process sequences of user actions	Stateful stream operations.
Data transformation	Convert records using context	Custom intermediate operation.
ETL pipelines	Group/process records incrementally	Reusable custom stream logic.

7. Example — fixed windows

Suppose you receive:

1 2 3 4 5 6 7 8 9

and want groups of three.

Using the built-in gatherer:

```
var result = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
    .gather(Gatherers.windowFixed(3))
    .toList();
```

Result:

[

```
[1, 2, 3],  
[4, 5, 6],  
[7, 8, 9]  
]
```

This is particularly useful for:

- database batch inserts
 - API request batching
 - message processing
 - ETL
 - bulk operations
-

8. Example — sliding windows

Suppose:

```
10 20 30 40 50
```

and you want windows of three:

```
[10, 20, 30]  
[20, 30, 40]  
[30, 40, 50]
```

A gatherer can express this directly:

```
var result = Stream.of(10, 20, 30, 40, 50)  
    .gather(Gatherers.windowSliding(3))  
    .toList();
```

This is particularly useful for **moving averages**.

For example:

Measurements:

```
10 20 30 40 50
```

3-value windows:

```
10 20 30 → average 20  
20 30 40 → average 30  
30 40 50 → average 40
```

9. Real-world example — moving average

Imagine a monitoring system receiving CPU utilization:

20, 40, 60, 80, 50, 70

You want a moving average using three measurements.

Conceptually:

[20,40,60] → 40
[40,60,80] → 60
[60,80,50] → 63.33
[80,50,70] → 66.67

A gatherer allows the windowing operation to be represented naturally:

```
var averages =  
    cpuMeasurements.stream()  
        .gather(Gatherers.windowSliding(3))  
        .map(window ->  
            window.stream()  
                .mapToDouble(Double::doubleValue)  
                .average()  
                .orElse(0))  
        .toList();
```

This is a good example of where a gatherer adds value because the operation requires **context from neighboring elements**.

10. Example — custom gatherer concept

A custom gatherer can maintain state.

Conceptually:

input element
↓
update state
↓
decide whether to emit
↓
send result downstream

For example, suppose you want:

Emit an element only when its value is greater than the previous element.

Input:

10, 15, 12, 20, 18, 25

Output:

15, 20, 25

A custom gatherer can maintain:

previous value

and compare every incoming element against that state.

This is awkward to express cleanly using only ordinary `map()/filter()` operations.

11. Stream Gatherers vs ordinary Stream API

Requirement	Standard Stream	Gatherer
filter	★ ★ ★ ★ ★	★
map	★ ★ ★ ★ ★	★
flatMap	★ ★ ★ ★ ★	★
sorted	★ ★ ★ ★ ★	★
distinct	★ ★ ★ ★ ★	★
Simple aggregation	★ ★ ★ ★ ★	★ ★
Fixed windows	★ ★ ★	★ ★ ★ ★ ★
Sliding windows	★ ★	★ ★ ★ ★ ★
Stateful intermediate operation	★ ★	★ ★ ★ ★ ★
Custom intermediate operation	★ ★	★ ★ ★ ★ ★
Neighbor-aware processing	★ ★	★ ★ ★ ★ ★
Reusable custom stream behavior	★ ★ ★	★ ★ ★ ★ ★

Simplicity	★★★★★	★★★
Learning curve	Low	Medium/High

12. Stream Gatherers vs Collectors

This distinction is important.

Aspect	Stream Gatherer	Collector
Position	Intermediate operation	Terminal operation
Produces stream elements	✓	✗ Usually produces final result
Stateful processing	✓	✓
Windowing	✓	✗ Not its primary purpose
Transform stream during processing	✓	✗
Final aggregation	☐ Possible but not primary purpose	✓ Excellent
Example	stream.gather(...)	stream.collect(...)
Best use	Custom intermediate processing	Accumulating final result

Think of it as:

```

Stream
  ↓
filter
  ↓
map
  ↓
GATHERER      ← custom intermediate processing
  ↓
map
  ↓
collect      ← final aggregation

```

13. Alternate features

Requirement	Alternative	When to prefer it
Simple transformation	map()	Almost always for one-to-one transformation.
Filtering	filter()	Simple predicate-based selection.
Flattening	flatMap()	Converting one element into multiple elements.
Final aggregation	collect() / Collectors	When you need a final result.
Simple stateful calculation	reduce()	When the operation naturally reduces the stream to one value.
Grouping	Collectors.groupingBy()	When grouping into a final map is the goal.
Fixed-size batches	Gatherers.windowFixed()	Prefer gatherer when available and appropriate.
Sliding windows	Gatherers.windowSliding()	Natural solution for overlapping windows.
Complex algorithm	Traditional for loop	When imperative code is significantly clearer.
Reactive event processing	Reactive Streams / Reactor	When you need asynchronous/back-pressure-oriented pipelines.
Parallel computation	Parallel Streams / Fork-Join	When task-level parallelism is the main requirement.

14. Gatherers vs traditional for loop

Aspect	Gatherer	for loop
Declarative style	★ ★ ★ ★ ★	★ ★
Simple logic	★ ★ ★	★ ★ ★ ★ ★
Complex state	★ ★ ★ ★	★ ★ ★ ★ ★

Reusability	★ ★ ★ ★ ★	★ ★
Stream composition	★ ★ ★ ★ ★	★ ★
Debugging	★ ★ ★	★ ★ ★ ★ ★
Learning curve	★ ★ ★	★ ★ ★ ★ ★
Batch/window operations	★ ★ ★ ★ ★	★ ★ ★
Fine-grained imperative control	★ ★ ★	★ ★ ★ ★ ★
Parallel stream integration	★ ★ ★ ★	★ ★
Best for	Reusable stream operations	Straightforward algorithms

Important: A gatherer isn't intended to eliminate for loops. If a loop is clearer, a loop is often the better engineering choice.

15. Performance considerations

Factor	Effect
Large streams	Gatherers can be useful.
Stateful processing	Gatherer can avoid awkward external mutable state.
Windowing	Can make implementation more natural.
Small streams	Performance benefits may be negligible.
Complex custom gatherer	Benchmark carefully.
Parallel streams	Correct parallel semantics are important.
Allocation-heavy gatherer	May reduce performance.
Simple map/filter pipeline	Don't replace it unnecessarily.

CPU-bound computation	Benchmark against alternatives.
I/O-bound operation	Gatherer usually won't solve the primary bottleneck.

Key principle:

Stream Gatherers primarily improve **expressiveness and composability**, not automatically performance.

16. When NOT to use Stream Gatherers

Don't use it when...	Better choice
You only need filtering	<code>filter()</code>
You only need transformation	<code>map()</code>
You only need flattening	<code>flatMap()</code>
You need a final sum	<code>sum()</code> / <code>reduce()</code>
You need a final grouping	<code>Collectors.groupingBy()</code>
The algorithm is highly imperative	for loop
The operation is easier to understand outside Streams	Regular method/loop
The team doesn't need custom stream operations	Standard Stream API
You are targeting Java without the feature	Standard Streams / custom utilities
The feature's preview status is unacceptable	Stable alternatives
You're using it only to make code "more functional"	Prefer the clearest implementation

17. Stream Gatherers vs alternatives — overall comparison

Feature	Best for	Complexity	Stateful	Intermediate	Final result
---------	----------	------------	----------	--------------	--------------

map()	Transforming elements	Low	No	✓	No
filter()	Selecting elements	Low	No	✓	No
flatMap()	Flattening	Medium	No	✓	No
reduce()	Reduction	Medium	Yes	✗	✓
Collectors	Aggregation/grouping	Medium	Yes	✗	✓
Stream Gatherer	Custom intermediate processing	Medium/High	Yes	✓	Not primarily
for loop	Arbitrary algorithms	Low/Medium	Yes	N/A	Flexible
Reactive Streams	Async/event streams	High	Yes	✓	Flexible

18. Final recommendation

Question	Recommendation
Should I learn it?	✓ Yes , especially if you work heavily with Java Streams and advanced data processing.
Should I use it everywhere?	✗ No.
Best use case	Custom stateful intermediate stream operations, windowing, batching, and sequence processing.
Best simple alternative	Standard Stream operations such as map, filter, flatMap, and collect.
Best alternative for complex imperative logic	Traditional for loop.
Best alternative for final aggregation	Collectors / reduce.

Biggest advantage	Allows developers to create reusable custom intermediate Stream operations .
Biggest disadvantage	Increased complexity and the fact that it was a second-preview feature in Java 23 .
Performance benefit?	☐ Not guaranteed. Benchmark real workloads.
Best real-time applications	Batch processing, rolling analytics, sensor data, event sequences, ETL, monitoring, and time-series processing.
Production recommendation for Java 23	Use cautiously because it was still a preview feature; for stable production systems, consider finalized versions or standard Stream constructs where practical.

One-line takeaway

Stream Gatherers are best thought of as an extension point for the Java Stream API: use them when `map()`/`filter()`/`collect()` become awkward because your operation needs state, windows, batching, neighboring elements, or custom intermediate behavior.

Java 23 — Deprecate Memory-Access Methods in `sun.misc.Unsafe`

Aspect	Details
Java Version	23
Year	2024
Feature	Deprecate the Memory-Access Methods in <code>sun.misc.Unsafe</code> for Removal
UniqueClassName	<code>UnsafeMemoryAccessDeprecation</code>
JEP	JEP 471
Status	Deprecated for removal
Primary target	Memory-access methods in <code>sun.misc.Unsafe</code>
Reason	Replace unsupported/unsafe memory-access mechanisms with standard, supported Java APIs
Main replacements	VarHandle API and Foreign Function & Memory (FFM) API

Important distinction	This JEP is primarily a migration/deprecation change , not a new programming feature.
Java 24+ direction	Runtime warnings were introduced for use of these Unsafe memory-access methods. (Oracle Docs)

Java 23 deprecated the memory-access methods in `sun.misc.Unsafe` for future removal because standard APIs have superseded them: `VarHandle` for many on-heap/atomic access scenarios and the FFM API for foreign/off-heap memory. ([OpenJDK Mail](#))

1. What exactly is being deprecated?

It is important to understand that **JEP 471 does not mean that all of `sun.misc.Unsafe` disappeared in Java 23.**

The focus is on its **memory-access methods**, which historically allowed Java libraries to perform operations outside normal Java safety guarantees.

Conceptually, older code could do things like:

```
Unsafe unsafe = ...;
```

```
long address = ...;
```

```
int value = unsafe.getInt(address);
unsafe.putInt(address, 123);
```

Or access object fields using offsets:

```
long offset = unsafe.objectFieldOffset(field);
```

```
int value = unsafe.getInt(object, offset);
```

These mechanisms are powerful, but they bypass many normal Java safety mechanisms.

JEP 471 encourages migration toward supported APIs, particularly **`VarHandle`** and **`FFM`**. ([OpenJDK Mail](#))

2. Why was Unsafe memory access deprecated?

Problem with Unsafe	Explanation
---------------------	-------------

Unsupported API	sun.misc.Unsafe is not intended as a general public application API.
Bypasses Java safety	It can perform operations that ordinary Java code cannot.
Memory corruption risk	Incorrect native/off-heap access can cause serious failures.
JVM crash potential	Incorrect low-level memory manipulation can potentially crash the JVM.
Poor abstraction	Code works with offsets and raw memory rather than meaningful layouts/types.
JDK-internal dependency	Applications become dependent on implementation details of the JDK.
Migration difficulty	Internal APIs can make applications fragile across Java releases.
Standard replacements exist	VarHandle and FFM provide supported APIs for many of these use cases.

The OpenJDK proposal explicitly states that the affected Unsafe methods have been superseded by **VarHandle** and **FFM**. ([OpenJDK Mail](#))

3. Advantages of the deprecation

Since this is a **deprecation/migration feature**, its "advantages" are primarily benefits to the Java ecosystem rather than new capabilities.

Advantage	Explanation	Benefit
Improves Java platform integrity	Encourages applications away from unsupported low-level APIs.	More predictable applications.
Encourages standard APIs	Moves developers toward VarHandle and FFM.	Better long-term compatibility.
Reduces dependency on JDK internals	Applications no longer need to depend on sun.misc.Unsafe for supported use cases.	Easier JDK upgrades.
Improves maintainability	Standard APIs provide more meaningful abstractions.	Easier for new developers to understand.

Better memory safety	FFM provides structured memory segments, layouts, and access mechanisms.	Reduces accidental memory errors.
Better type modeling	MemoryLayout describes the structure of foreign memory.	Less offset arithmetic.
Clearer concurrency semantics	VarHandle explicitly models memory access modes and atomic operations.	Better concurrent programming.
Better ecosystem compatibility	Libraries can migrate away from JDK-internal APIs.	Frameworks become more future-proof.
Supports gradual migration	Deprecation gives library developers time to migrate before eventual removal.	Less disruptive transition.
Improves portability	Standard APIs are better suited to evolving JVM implementations.	Reduced reliance on implementation details.

4. Disadvantages

Disadvantage	Explanation	Practical impact
Migration effort	Existing libraries may contain substantial Unsafe usage.	Refactoring can be expensive.
Performance uncertainty	Some libraries adopted Unsafe specifically for highly optimized operations.	Must benchmark replacements.
API differences	VarHandle and FFM are not drop-in replacements for every Unsafe operation.	Code changes may be substantial.
Legacy compatibility	Libraries supporting old JDKs may need multiple implementations.	Multi-release JARs or compatibility layers may be necessary.

Complex migration for frameworks	Serialization, collections, concurrency, and memory libraries may have deep Unsafe integration.	Architecture-level changes may be required.
Short-term complexity	Developers may need to understand both old and new approaches during migration.	Increased maintenance during transition.
FFM itself requires expertise	Foreign memory and native interoperability are advanced topics.	Learning curve remains.
Some Unsafe use cases are specialized	Not every historical use has a simple replacement.	Special cases require careful redesign.
Warnings in newer JDKs	Continued use becomes increasingly visible during migration.	Legacy dependencies may generate runtime warnings.

5. When to use what?

The key question isn't really:

"When should I use the deprecation?"

Instead, ask:

"What should replace the Unsafe operation I currently have?"

Requirement	Recommended API
Atomic access to an object field	VarHandle
Atomic access to array elements	VarHandle
Volatile field access	VarHandle
Compare-and-set operations	VarHandle
Acquire/release memory semantics	VarHandle

Access to structured foreign memory	FFM API
Native/off-heap memory	FFM API
Call C/native functions	FFM API
Access a native library	FFM API
Ordinary Java object field access	Normal Java / Reflection where appropriate
Ordinary array access	Normal Java arrays
Dynamic method invocation	MethodHandle / MethodHandles
Bytecode generation	Class-File API / Byte Buddy / ASM depending on requirement

6. Unsafe → VarHandle

One of the most important migration paths is:

Unsafe memory access



VarHandle

For example, instead of relying on Unsafe to perform atomic field operations, you can use:

```
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

class Counter {
    private volatile int value;

    private static final VarHandle VALUE;

    static {
        try {
            VALUE = MethodHandles.lookup()
                .findVarHandle(Counter.class, "value", int.class);
        } catch (ReflectiveOperationException e) {
            throw new ExceptionInInitializerError(e);
        }
    }
}
```

```

    public boolean increment(int expected) {
        return VALUE.compareAndSet(this, expected, expected + 1);
    }
}

```

The important difference is that VarHandle provides a **standardized access abstraction** rather than exposing raw memory offsets.

Java's VarHandle supports access modes such as ordinary, opaque, acquire/release, and volatile access, along with atomic operations where supported. ([Oracle Docs](#))

7. Unsafe → FFM API

For off-heap or native memory, the migration is conceptually:

```

Unsafe raw memory
  ↓
MemorySegment
  +
MemoryLayout
  +
VarHandle

```

For example:

```

import java.lang.foreign.Arena;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;

try (Arena arena = Arena.ofConfined()) {

    MemorySegment memory =
        arena.allocate(ValueLayout.JAVA_INT);

    memory.set(ValueLayout.JAVA_INT, 0, 123);

    int value =
        memory.get(ValueLayout.JAVA_INT, 0);

    System.out.println(value);
}

```

The FFM API provides structured access to memory outside the Java runtime and is intended to avoid the brittleness and danger associated with JNI for many native interoperability tasks. ([Oracle Docs](#))

8. Real-time use cases

Real-world use case	Historical Unsafe usage	Recommended modern alternative
High-performance concurrent counter	Atomic field access	VarHandle
Lock-free data structure	CAS operations	VarHandle
Concurrent queue	Volatile/CAS field access	VarHandle
Off-heap database buffer	Raw memory addresses	FFM MemorySegment
Native database integration	Native pointers	FFM
C library integration	JNI/Unsafe-style memory	FFM
Serialization framework	Field offsets/object manipulation	Modern reflection/MethodHandles/VarHandles, depending on requirement
High-performance networking library	Direct memory access	FFM / supported buffer APIs
Native image/data processing	Raw native memory	FFM
Memory-mapped/native structures	Raw addresses	FFM + MemoryLayout
Atomic array operations	Unsafe array offsets	VarHandle
Custom lock-free algorithm	CAS/volatile operations	VarHandle

9. Example — lock-free counter

Old style

A library might historically use Unsafe to perform CAS:

```
unsafe.compareAndSwapInt(
```

```
this,  
VALUE_OFFSET,  
expected,  
updated  
);
```

Modern style

```
VALUE.compareAndSet(  
    this,  
    expected,  
    updated  
);
```

Conceptually:

Old:

```
Object  
↓  
Unsafe  
↓  
memory offset  
↓  
CAS
```

Modern:

```
Object  
↓  
VarHandle  
↓  
named field/access mode  
↓  
CAS
```

This is clearer and avoids exposing raw memory offsets to application code.

10. Example — off-heap memory

Old conceptual model

```
allocate memory  
↓  
raw address  
↓
```

Unsafe.getInt(address)
↓
Unsafe.putInt(address, value)

Modern FFM model

Arena
↓
MemorySegment
↓
MemoryLayout / ValueLayout
↓
get() / set()

Example:

```
try (Arena arena = Arena.ofConfined()) {  
    MemorySegment segment =  
        arena.allocate(1024);  
  
    segment.set(  
        ValueLayout.JAVA_INT,  
        0,  
        42  
    );  
  
    int value = segment.get(  
        ValueLayout.JAVA_INT,  
        0  
    );  
}
```

The FFM API gives the memory a structured lifetime and layout rather than making the application manually manage a raw address.

11. Why FFM is safer than raw Unsafe memory

Aspect	Unsafe raw memory	FFM
Memory address abstraction	Raw/low-level	MemorySegment
Memory lifetime	Developer-managed	Explicit segment/arena lifetime
Layout description	Usually manual	MemoryLayout
Type information	Limited	Stronger

Bounds/lifetime model	Easy to misuse	API provides checks
Native function calls	Low-level	Linker / downcall handles
Standard Java API	✗	✓
Supported long-term API	✗	✓
Memory safety	Low	Higher
Learning curve	High	High, but structured

FFM still contains **restricted methods** because native interoperability is inherently capable of unsafe operations; it is not "magically safe" in every scenario. ([Oracle Docs](#))

12. When NOT to use Unsafe

Situation	Use Unsafe?	Better option
Normal object access	✗	Normal Java
Normal array access	✗	Java arrays
Atomic field operations	✗	VarHandle
CAS operations	✗	VarHandle
Off-heap memory	✗	FFM
Native function calls	✗	FFM
Native structures	✗	FFM + MemoryLayout
Dynamic invocation	✗	MethodHandle
Bytecode generation	✗	Class-File API / ASM / Byte Buddy
Ordinary application code	✗	Standard Java APIs
New library development	✗ Generally	Supported standard APIs
Existing legacy library	?	Migrate incrementally

13. When might Unsafe still appear?

This is an important practical distinction.

Scenario	Recommendation
New application	Do not introduce Unsafe memory access.
New library	Prefer supported APIs.
Existing high-performance library	Analyze and migrate carefully.
Third-party dependency uses Unsafe	Upgrade the dependency if possible.
No direct replacement exists	Isolate the usage and document it.
Performance-critical legacy code	Benchmark replacement before migration.
Java 8 compatibility requirement	You may need compatibility strategies such as multi-release JARs.
Framework with many JDK versions	Design a version-specific implementation where necessary.

The OpenJDK discussion explicitly recognized that some libraries have historically relied on Unsafe, including for specialized high-concurrency techniques, so migration isn't always a simple search-and-replace exercise. ([OpenJDK Mail](#))

14. Unsafe vs VarHandle vs FFM

Capability	sun.misc.Unsafe	VarHandle	FFM
Standard public API	✗	✓	✓
Object field access	✓	✓	✗
Atomic field access	✓	✓	✗
CAS	✓	✓	✗

Memory ordering	Low-level	★★★★★	Via appropriate handles / layouts
Raw off-heap memory	✓	✗	✓
Native function calls	✗ / indirect	✗	✓
Native structures	Difficult	✗	★★★★★
Memory lifetime management	Manual	JVM managed	Explicit segments/arenas
Type/layout description	Weak	Stronger	★★★★★
Long-term recommendation	✗	✓	✓
Main purpose	Legacy low-level access	Variable/atomic access	Foreign memory/native interoperability

15. Alternate features

Alternative feature	Best use	Replaces which Unsafe usage?
VarHandle	Atomic/volatile/ordered variable access	Field/array memory-access operations
Foreign Function & Memory API	Native memory and native functions	Raw off-heap memory and native interoperability
MemorySegment	Managed foreign memory	Raw memory addresses
MemoryLayout	Structured native-memory representation	Manual offset/size calculations
MethodHandle	Dynamic method invocation	Some low-level invocation use cases
Reflection	Runtime metadata/access	Some object introspection use cases
Class-File API	Class-file manipulation	Bytecode-related Unsafe work
Byte Buddy	Runtime class	Framework-specific dynamic-

	generation/instrumentation	class use cases
JNI	Native interoperability	Legacy alternative, but FFM is generally preferred for new native interop

Oracle's migration guidance specifically identifies **VarHandle** and **FFM** as the principal supported replacements for the deprecated memory-access methods. ([Oracle Docs](#))

16. Migration decision table

Your current Unsafe operation	Recommended migration
getInt(Object, offset)	VarHandle where accessing a Java variable/field
putInt(Object, offset, value)	VarHandle
compareAndSwapInt(...)	VarHandle.compareAndSet(...)
getLongVolatile(...)	VarHandle.getVolatile(...)
putLongVolatile(...)	VarHandle.setVolatile(...)
getObjectVolatile(...)	VarHandle.getVolatile(...)
Raw native memory address	MemorySegment
Native structure	MemoryLayout + MemorySegment
Native function invocation	FFM Linker
Native pointer manipulation	FFM address layouts/segments
Atomic access to foreign memory	FFM + appropriate VarHandle
Dynamic invocation	MethodHandle
Class generation	Class-File API / Byte Buddy / ASM

17. Performance considerations

A common concern is:

"If I migrate away from Unsafe, will my application become slower?"

The answer is:

Not necessarily.

The modern APIs are designed for high-performance JVM use cases.

Concern	Recommendation
VarHandle performance	Benchmark the actual hot path.
FFM performance	Benchmark against the existing native/JNI implementation.
Lock-free algorithms	Benchmark contention patterns, not just single-thread latency.
Off-heap processing	Measure allocation, access, copying, and native-call costs.
Serialization	Benchmark realistic object graphs.
Networking	Benchmark complete request/response throughput.
Microbenchmarking	Use JMH , not <code>System.nanoTime()</code> around one operation.

The goal of JEP 471 is **not to sacrifice performance for safety**; it is to move common low-level operations onto supported APIs. The exact performance characteristics should still be verified for each workload.

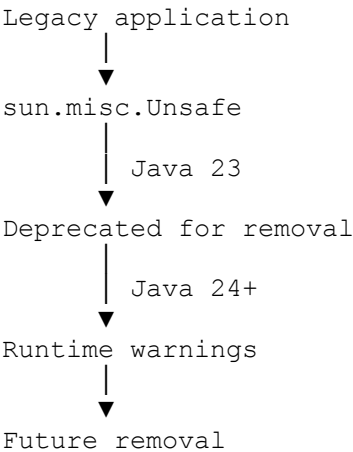
18. Impact on existing applications

Application situation	Impact
No Unsafe usage	☐ No impact
Uses only unrelated Unsafe functionality	☐ Check whether those APIs are affected by JEP 471
Direct memory-access methods	☐ Migration required eventually
Third-party dependency uses Unsafe	☐ Upgrade dependency/check vendor support
High-performance framework	☐ Detailed migration/testing required
Java agent/instrumentation library	☐ Analyze its specific usage

Serialization library	❓ Potentially significant
Lock-free data structure library	❓ Benchmark migration
Native-memory library	❓ Strong candidate for FFM migration

19. Java 23 → Java 24+ migration direction

The deprecation is part of a larger migration path:



Oracle's migration documentation states that the affected methods were terminally deprecated in JDK 23, and Java 24 introduced runtime warnings on their first use. ([Oracle Docs](#))

Therefore, **Java 23 should be treated as a migration signal rather than a reason to start new Unsafe code.**

20. Final comparison

Category	Unsafe	VarHandle	FFM
Safety	❓ Low	❓ High	❓ Higher
Standard API	❓ No	❓ Yes	❓ Yes
Object field access	❓	❓	✗
Atomic operations	❓	❓	❓

Raw memory	?	×	?
Native memory	?	×	?
Native function calls	?	×	?
Type/layout abstraction	?	?	??
Long-term recommendation	? Migrate	?	?
Learning difficulty	High	Medium/High	High
JDK compatibility	? Poor long-term	? Good	? Good

Final recommendation

Question	Recommendation
Should I use sun.misc.Unsafe in new code?	✗ No, especially for memory-access operations.
Should existing Unsafe code be migrated?	✓ Yes, plan migration.
Best replacement for atomic/volatile field access?	VarHandle
Best replacement for off-heap/native memory?	Foreign Function & Memory API
Best replacement for native function calls?	FFM API
Should I replace Unsafe blindly?	✗ No. Understand the exact operation and benchmark the replacement.
Is this a performance feature?	✗ No. It is primarily a platform-integrity and migration feature .
Main benefit	Moves Java developers from unsupported internal memory APIs toward supported, standardized APIs.
Main disadvantage	Existing low-level libraries may require significant migration and performance validation.
Best real-world approach	New code → VarHandle/FFM; existing code → inventory, benchmark, migrate, test, and

	remove Unsafe dependencies.
--	-----------------------------

One-line takeaway: Java 23's JEP 471 is essentially a "stop building new systems on Unsafe memory access" signal: use **VarHandle** for controlled variable/atomic access and **FFM** for foreign/off-heap memory and native interoperability. ([OpenJDK Mail](#))

Java 23 — ZGC Generational Mode by Default

Aspect	Details
Java Version	23
Year	2024
Feature	ZGC Generational Mode by Default
UniqueClassName	GenerationalZGC
JEP	474 — ZGC: Generational Mode by Default
Status	Final change in Java 23
Main component	Z Garbage Collector (ZGC)
Change	Generational ZGC became the default ZGC mode
Key idea	ZGC separates objects into young and old generations , allowing the collector to focus more efficiently on recently created objects.
Main objective	Improve application throughput and reduce garbage-collection overhead while retaining ZGC's low-pause characteristics.

Important: Java 23 did **not** make ZGC the default garbage collector for all Java applications. Rather, **when ZGC is selected, generational mode became the default.**

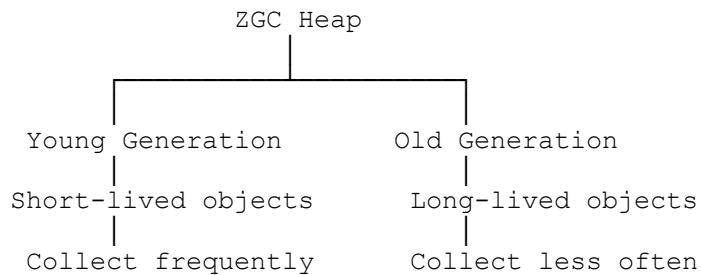
1. What is Generational ZGC?

Traditional ZGC was designed as a low-latency garbage collector capable of handling very large heaps.

Generational ZGC adds the generational hypothesis:

Most newly created objects die young.

Instead of treating all objects similarly, the heap is conceptually divided into:



For example, a web request might create:

```
OrderRequest request = parseRequest();
ValidationResult result = validate(request);
Response response = createResponse(result);
```

Many temporary objects associated with the request may become unreachable shortly afterward.

Generational ZGC can concentrate collection effort on these **young objects**, while avoiding unnecessary work on objects that have already survived for a long time.

2. How it differs from non-generational ZGC

Aspect	Non-Generational ZGC	Generational ZGC
Generations	No explicit young/old distinction	Young + old
Short-lived objects	Collected as part of general ZGC processing	Specifically targeted
Long-lived objects	Treated as part of the overall heap	Can be treated separately
Allocation-heavy workloads	Good	Potentially better
GC efficiency	Good	Improved for many workloads
Low-pause design	✓	✓

Large heaps	✓	✓
Complexity	Lower conceptually	Higher internally
Java 23 ZGC default	✗	✓

3. Advantages

Advantage	Explanation	Practical benefit
Better handling of short-lived objects	Young objects can be collected more frequently.	Less unnecessary scanning of long-lived objects.
Lower GC overhead for many workloads	Generational collection can focus on regions with high amounts of garbage.	More CPU available for application work.
Retains ZGC's low-pause goal	Generational collection is designed within ZGC's concurrent architecture.	Suitable for latency-sensitive applications.
Good for allocation-heavy applications	Applications creating many temporary objects benefit from young-generation collection.	Potentially better throughput.
Better scalability for large heaps	ZGC is designed for large heap sizes.	Useful for large enterprise applications.
Improved throughput	Less work may be required to identify/reclaim garbage.	More useful application CPU time.
Automatic generational behavior	Developers don't have to manually enable generational mode when using ZGC in Java 23.	Simpler ZGC configuration.
Suitable for modern Java applications	Modern frameworks often create many temporary objects.	Good fit for high-allocation workloads.
Reduced need for manual GC tuning	ZGC aims to provide good behavior with relatively little configuration.	Easier operations.

Good latency/throughput balance	Combines low-pause goals with generational collection.	Useful for latency-sensitive services.
--	--	--

4. Disadvantages

Disadvantage	Explanation	Impact
Not automatically better for every workload	GC behavior depends heavily on allocation rate, heap size, object lifetime, and workload characteristics.	Benchmark before switching.
Requires sufficient heap resources	ZGC is generally targeted at applications where memory/latency requirements justify it.	May be excessive for small applications.
GC implementation is sophisticated	Generational ZGC involves complex concurrent GC mechanisms.	Diagnosis can require JVM expertise.
Potential GC CPU overhead	Concurrent GC consumes CPU while the application runs.	Important on CPU-constrained systems.
More complex GC logs/diagnostics	Understanding ZGC behavior requires familiarity with JVM GC logging.	Higher operational learning curve.
Not a replacement for fixing memory leaks	Generational GC cannot reclaim objects that remain reachable accidentally.	Memory leaks still require application fixes.
Large object behavior requires consideration	Very large allocations can behave differently from ordinary short-lived objects.	Application-specific tuning may be necessary.
Not always faster than G1	G1 can be an excellent choice for many general-purpose workloads.	Don't select ZGC purely because it is newer.
Small heap applications	ZGC's strengths become	G1 or Serial may be simpler.

may see little benefit	more valuable as latency/heap requirements increase.	
Operational compatibility	Existing GC tuning options may need reevaluation when changing collectors.	Test deployment configuration.

5. When should you use Generational ZGC?

Situation	Use?	Reason
Large Java heap	✓ Yes, consider it	ZGC is designed for large heaps.
Very low latency requirement	✓ Yes	ZGC targets low pause times.
High allocation rate	✓ Yes	Generational collection can efficiently process short-lived objects.
Large Spring Boot service with heavy object creation	✓ Potentially	Many request-scoped objects can be short-lived.
High-throughput API	✓ Potentially	Useful when GC latency affects throughput/SLAs.
Real-time-ish data processing	✓ Potentially	Low GC pauses can be valuable.
Large in-memory cache	✓ Potentially	Low-latency collection can be useful.
Interactive application with strict latency	✓ Potentially	Pause-time sensitivity favors ZGC.
Small CLI application	✗ Usually no	ZGC may be unnecessary.
Small heap	✗ Usually no	Simpler collectors may be more appropriate.
CPU-constrained environment	🔍 Benchmark	Concurrent GC consumes CPU resources.
Simple CRUD application	🔍 Usually G1 is sufficient	ZGC may not provide enough additional value.

Application already meets latency/throughput targets	✗ Don't change without reason	Avoid unnecessary tuning.

6. Real-time use cases

Real-time use case	Typical workload	Why Generational ZGC can help
High-throughput REST API	Thousands/millions of requests	Each request may generate many temporary objects.
E-commerce platform	Orders, carts, payments	Request processing creates short-lived objects.
Trading platform	Continuous market-data processing	Low GC pauses can be important.
Banking transaction engine	Large transaction volume	High allocation + latency sensitivity.
Gaming backend	Large number of concurrent sessions	Many temporary request/session objects.
Messaging platform	High message throughput	Message processing can create short-lived objects.
Big-data processing	Large data transformations	High allocation rate and large heaps.
Caching service	Large in-memory datasets	Large heap + latency requirements.
Real-time analytics	Continuous event processing	Many temporary intermediate objects.
IoT platform	Large volumes of sensor events	Continuous object allocation.
Search service	Query parsing/results	Many temporary query-processing objects.
Microservices at large scale	High request rates	Low latency can improve tail-latency SLAs.

7. Example — high-throughput REST API

Imagine:

```
@PostMapping("/orders")
public OrderResponse createOrder(OrderRequest request) {

    Customer customer = customerService.find(request.customerId());

    Order order = orderService.create(
        customer,
        request.items()
    );

    return OrderResponse.from(order);
}
```

A single request can create many temporary objects:

HTTP request
↓
DTOs
↓
Validation objects
↓
Collections
↓
Database objects
↓
Mapping objects
↓
Response DTO

Across millions of requests:

Request 1 → many temporary objects
Request 2 → many temporary objects
Request 3 → many temporary objects
...

Generational ZGC is designed to take advantage of the fact that many of these objects become unreachable quickly.

8. Example — temporary object allocation

Consider:

```

for (Transaction transaction : transactions) {

    TransactionSummary summary =
        createSummary(transaction);

    publish(summary);
}

```

If TransactionSummary objects don't survive for long:

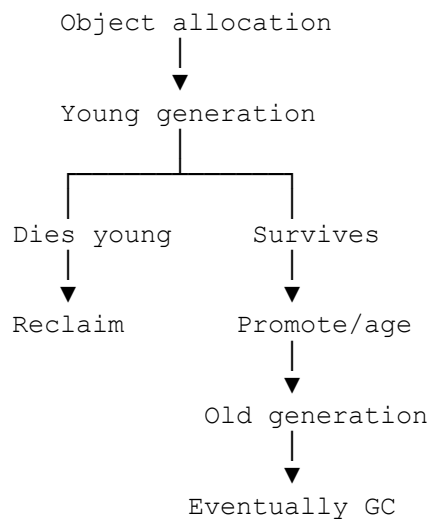
```

Create
↓
Use
↓
Become unreachable
↓
Young generation
↓
Reclaim

```

Long-lived objects such as application configuration, caches, and persistent domain structures can remain in the older part of the heap.

9. Generational ZGC conceptual lifecycle



The key optimization is:

Don't spend the same amount of GC effort on objects that are almost certainly dead and objects that have survived for a long time.

10. Generational ZGC vs G1

This is the most important practical comparison for many enterprise Java applications.

Aspect	Generational ZGC	G1 GC
Generational	✓	✓
Low-latency focus	★★★★★	★★★★
Large heaps	★★★★★	★★★★
Throughput	★★★★	★★★★★
Pause-time focus	★★★★★	★★★★
Mature/default general-purpose choice	★★★★	★★★★★
Configuration simplicity	★★★★	★★★★
Small/medium applications	★★★	★★★★★
Large latency-sensitive application	★★★★★	★★★★
High allocation workload	★★★★★	★★★★
Default GC in standard HotSpot	✗	Yes, G1
Best reason to choose	Low latency + large/high-allocation workload	General-purpose balance

Important: Generational ZGC being the default *mode for ZGC* does **not** mean it replaced G1 as the JVM's default garbage collector.

11. Generational ZGC vs other collectors

Collector	Best suited for	Latency	Throughput	Complexity
Serial GC	Small/simple	★★	★★	★★★★★

	applications			
Parallel GC	Throughput-oriented workloads	★★	★★★★★	★★★★
G1 GC	General-purpose server applications	★★★★	★★★★★	★★★★
Generational ZGC	Large heaps + low latency + high allocation	★★★★★	★★★★	★★★
Shenandoah	Low-latency large-heap applications	★★★★★	★★★★	★★★

12. When NOT to use Generational ZGC

Situation	Better choice	Reason
Tiny application	Serial/G1	ZGC is unnecessary complexity.
Small heap	G1/Serial	ZGC's large-heap advantages may not matter.
CPU-constrained container	Benchmark G1/Parallel	Concurrent GC consumes CPU.
Pure throughput workload	Parallel/G1	Throughput may be the primary objective.
Application already performs well on G1	Stay with G1	Don't optimize without evidence.
Short-lived CLI utility	Serial/G1	Simpler collector is preferable.
Memory leak	Fix application	Changing GC doesn't solve reachability problems.
No latency SLA	G1/Parallel may be sufficient	ZGC's latency advantage may not justify it.
Very low allocation workload	Benchmark alternatives	Generational behavior may provide little benefit.

Unknown workload	Profile first	Collector choice should be evidence-based.

13. Configuration

In Java 23, if you select ZGC:

```
java -XX:+UseZGC -jar application.jar
```

Generational mode is the default ZGC mode in Java 23.

Therefore, you generally don't need a separate option just to request generational ZGC.

For comparison, an older/non-generational configuration could historically involve explicitly disabling generational behavior:

```
java -XX:+UseZGC -XX:-ZGenerational -jar application.jar
```

For Java 23+, the important point is that **generational ZGC is the normal/default mode when ZGC is selected**.

14. Monitoring Generational ZGC

Use GC logging when evaluating it:

```
java \
-Xlog:gc* \
-XX:+UseZGC \
-jar application.jar
```

Monitor:

Metric	Why
GC pause time	Verify latency goals.
Allocation rate	Determines pressure on young generation.
Heap occupancy	Understand memory utilization.
CPU utilization	Concurrent GC consumes CPU.
Tail latency	Important for APIs and real-time services.

Throughput	Ensure GC isn't reducing application throughput.
Full GC behavior	Investigate unusual memory pressure.
Native memory	ZGC has its own memory-management overhead.

15. Performance tuning strategy

Don't begin with:

"ZGC is newer, therefore it must be faster."

Instead:

```

Application
  ↓
Measure baseline
  ↓
Profile allocation
  ↓
Measure GC behavior
  ↓
Try G1
  ↓
Try Generational ZGC
  ↓
Compare:
  • throughput
  • p95 latency
  • p99 latency
  • GC CPU
  • memory usage
  ↓
Choose winner

```

For a latency-sensitive service, **p99/p999 latency** can be more important than average latency.

16. Alternate features / collectors

Alternative	When to use	Compared with Generational ZGC
G1 GC	General-purpose server applications	Usually the first alternative to evaluate.

Parallel GC	Maximum throughput	Better when pause time is less important.
Serial GC	Small applications	Much simpler and lower overhead for small heaps.
Shenandoah GC	Low-latency large heaps	Another low-pause alternative.
Epsilon GC	Specialized testing/benchmarking	No garbage collection; not for normal production workloads.
Application-level allocation reduction	Excessive object creation	Can help regardless of collector.
Object pooling	Very specific high-allocation cases	Often unnecessary; benchmark carefully.
Primitive/data-oriented structures	Allocation-heavy numerical processing	Reduces GC pressure at the application level.

17. Generational ZGC vs application optimization

Changing the garbage collector isn't always the best solution.

Problem	First thing to investigate
Millions of temporary objects	Reduce unnecessary allocations.
Excessive boxing	Use primitives where appropriate.
Huge temporary collections	Reduce intermediate collection creation.
String allocation	Review string-building/parsing strategy.
Memory leak	Find unintended object retention.
Long GC pauses	Evaluate collector and heap configuration.
High CPU from GC	Analyze allocation rate and collector choice.
High p99 latency	Analyze GC + application + database/network latency.
Large heap requirement	Analyze retained object graph and cache size.

18. Decision table

Application profile	Recommended collector to evaluate
Small CLI	Serial / G1
Standard Spring Boot service	G1 first
Large Spring Boot service with strict latency SLA	G1 vs Generational ZGC benchmark
Large in-memory application	Generational ZGC / G1
High allocation + low latency	Generational ZGC
Maximum throughput	Parallel / G1
Huge heap + strict latency	Generational ZGC
CPU-constrained container	Benchmark G1/Parallel vs ZGC carefully
Small heap + low traffic	G1 / Serial
Real-time analytics	Generational ZGC candidate
Large trading/market-data system	Generational ZGC candidate
Memory-leak problem	Fix application first

19. Final comparison

Category	Generational ZGC assessment
Low latency	☑ Excellent
Large heaps	☑ Excellent
High allocation rate	☑ Excellent candidate
Short-lived objects	☑ Excellent
General-purpose applications	☑ Good, but compare with G1

Small applications	❑ Usually unnecessary
Maximum throughput	❑ Benchmark against Parallel/G1
CPU-constrained environment	❑ Benchmark carefully
Memory leak solution	❑ No
Ease of operation	❑ Relatively straightforward
JVM tuning required	❑ Application-specific
Java 23 status	❑ Generational mode is default when ZGC is selected

Final recommendation

Question	Answer
Should I use Generational ZGC?	✓ Consider it for large heaps, high allocation rates, and strict latency requirements.
Is it automatically better than G1?	✗ No. Benchmark both using your real workload.
Is ZGC the default GC in Java 23?	✗ No. G1 remains the default general-purpose collector.
What became default?	Generational mode became the default when ZGC is selected.
Biggest advantage	Excellent combination of low-latency garbage collection and efficient handling of short-lived objects.
Biggest disadvantage	ZGC can consume additional CPU/resources and may provide little benefit for small or non-latency-sensitive applications.
Best real-time use cases	High-throughput APIs, trading systems, large in-memory services, real-time analytics, messaging, and large event-processing systems.
Best alternative	G1 GC for general-purpose applications; Shenandoah is another low-latency alternative.
Performance rule	Measure p95/p99 latency, throughput, allocation rate, GC CPU, and heap usage before and after changing collectors.

One-line takeaway: Java 23 made Generational ZGC the standard mode of ZGC, giving ZGC a young/old-generation model designed to handle short-lived objects more efficiently while preserving its low-latency focus.

Java 23 — Module Import Declarations (Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Module Import Declarations (Preview)
UniqueClassName	ModuleImportDeclarations
JEP	JEP 476
Status in Java 23	Preview
Main purpose	Allow a Java source file to import all accessible types exported by a module with a single module import declaration.
Syntax	import module java.sql;
Problem addressed	Reduces the need to know and write many individual imports when using APIs from a module.
Relationship to ordinary imports	It supplements, rather than completely replaces, import type and import static declarations.

1. What is Module Import Declarations?

Normally, if an application uses several classes from a module, you might write:

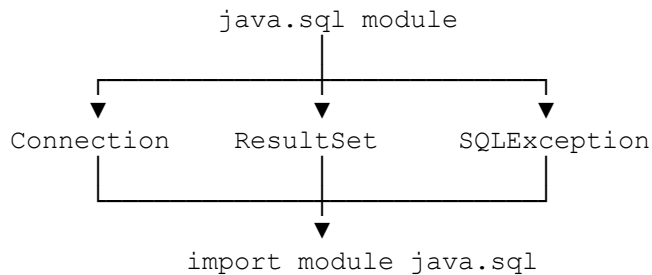
```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

With a module import, Java 23 allows:

```
import module java.sql;
```

This makes the **accessible exported types of the java.sql module** available for simple-name use.

Conceptually:



This is particularly useful when working with a module that exposes a large API.

2. Basic example

Traditional imports

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class CustomerRepository {

    public void findCustomer() throws Exception {

        Connection connection =
            DriverManager.getConnection(
                "jdbc:postgresql://localhost/customerdb");

        PreparedStatement statement =
            connection.prepareStatement(
                "SELECT * FROM customer");

        ResultSet resultSet =
            statement.executeQuery();

        // ...
    }
}
```

Module import

```

import module java.sql;

public class CustomerRepository {

    public void findCustomer() throws Exception {

        Connection connection =
            DriverManager.getConnection(
                "jdbc:postgresql://localhost/customerdb");

        PreparedStatement statement =
            connection.prepareStatement(
                "SELECT * FROM customer");

        ResultSet resultSet =
            statement.executeQuery();

        // ...
    }
}

```

The second version avoids explicitly listing every imported type.

3. What does import module actually import?

An important detail is that:

```
import module java.sql;
```

does **not** mean:

```
import java.sql.*;
```

It is a module-level mechanism.

The compiler considers the types exported by the module and its accessible dependencies when resolving simple type names.

This distinction matters because Java's module system operates at a higher level than packages.

Traditional mechanism	Module import
import java.sql.Connection;	Imports one type
import java.sql.*;	Imports types from one package
import module java.sql;	Makes accessible exported types from the

	module available
Scope	Package/type-oriented

4. Advantages

Advantage	Explanation	Practical benefit
Fewer import statements	Multiple API types can become available through one declaration.	Less boilerplate.
Convenient for large modules	Modules can expose many packages/types.	Easier source-code authoring.
Reduces API knowledge required initially	Developers don't need to remember every package containing required types.	Faster prototyping and learning.
Works at module level	Aligns imports with Java's module system.	Better conceptual relationship between imports and modules.
Useful for exploratory code	Quickly access an entire module API.	Excellent for experiments and examples.
Useful for teaching	Beginners can focus on API usage instead of import management.	Cleaner educational examples.
Reduces repetitive imports	Especially when many types from one module are used.	Cleaner source files.
Works with modular applications	Designed specifically around JPMS modules.	Natural fit for modular Java applications.
Supports API evolution	Adding another required type from the same module may not require another import.	Less source churn.
Improves readability in some contexts	A reader can immediately see that a module's API is being used.	Useful for module-oriented code.

5. Disadvantages

Disadvantage	Explanation	Practical impact
Preview feature	Java 23 requires preview support.	Not suitable for projects avoiding preview features.
Potential name conflicts	Importing many types can create ambiguous simple names.	Explicit imports may become necessary.
Less precise dependency visibility	One module import can hide exactly which types are being used.	Some developers may prefer explicit imports.
Requires module-system knowledge	Developers must understand JPMS concepts.	Learning curve for non-modular projects.
Not necessarily useful for small APIs	One or two imports don't justify module import syntax.	Can become unnecessary abstraction.
Third-party module support depends on modularity	The feature is most natural when libraries are properly modularized.	Legacy/classpath libraries may not benefit as much.
IDE/tooling considerations	Tooling needs to understand preview syntax.	Verify IDE/build support.
Migration concerns	Preview syntax may change.	Future Java upgrades could require source changes.
Wildcard-like effect	Developers may accidentally make more names available than expected.	Name resolution can become less obvious.
Can hide source dependencies	A reader may need to inspect the module to know where a type originates.	Slightly less explicit code.

6. When should you use it?

Situation	Use?	Reason
Prototyping with a large JDK module	✓ Yes	Reduces import boilerplate.
Educational examples	✓ Yes	Keeps examples focused on the API.
Small demo programs	✓ Potentially	Simple source code.
Using many classes from one module	✓ Yes	Strongest practical benefit.
Modular application with well-defined dependencies	✓ Potentially	Fits naturally with JPMS.
Generated source code	✓ Potentially	Can reduce generated import lists.
Large API exploration	✓ Yes	Convenient while discovering APIs.
Only one or two types needed	✗ Usually no	Explicit imports are clearer.
Public library with strict explicit dependencies	🔍 Evaluate	Explicit imports may improve readability.
Many modules with overlapping type names	✗ Usually no	Name conflicts become more likely.
Non-modular legacy application	🔍 Limited benefit	The module system may not be central to the application.
Production project avoiding preview features	✗ No	It was preview in Java 23.

7. Real-time use cases

Real-time use case	Example	Benefit
JDBC application	Many java.sql types	One module import can replace numerous imports.
Desktop application	Many JavaFX module types	Convenient when a module exposes many APIs.
XML processing	Multiple XML-related APIs	Reduces import boilerplate.
Networking application	Multiple networking-	Easier API exploration.

	related APIs	
Educational application	Teaching Java modules	Makes examples shorter.
Proof of concept	Rapidly testing a JDK API	Faster source authoring.
Code generation	Generated source using many types from one module	Less generated boilerplate.
Internal enterprise application	Large modular JDK/library APIs	Can simplify imports.
Framework examples	Demonstrating multiple classes from one module	Cleaner documentation.
API experimentation	Exploring unfamiliar modules	Don't need to identify every package initially.

8. Example — JDBC

Suppose an application uses:

```
Connection
DriverManager
PreparedStatement
ResultSet
SQLException
Statement
```

Traditional:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

With Java 23 module import:

```
import module java.sql;
```

Then:

```
public class CustomerRepository {
```

```

public List<Customer> findAll()
    throws SQLException {

    Connection connection =
        DriverManager.getConnection(URL);

    PreparedStatement statement =
        connection.prepareStatement(
            "SELECT id, name FROM customer");

    ResultSet resultSet =
        statement.executeQuery();

    // ...
}
}

```

This is a good use case because JDBC applications frequently use several types from the `java.sql` module.

9. Module import vs package wildcard

This is one of the most important distinctions.

Feature	Example	What it covers
Single-type import	<code>import java.sql.Connection;</code>	One type
Package wildcard	<code>import java.sql.*;</code>	Types in <code>java.sql</code> package
Module import	<code>import module java.sql;</code>	Accessible exported types from the module
Static import	<code>import static java.lang.Math.PI;</code>	Static member
Static wildcard	<code>import static java.lang.Math.*;</code>	Static members

For example:

```
import java.sql.*;
```

is **package-level**.

Whereas:

```
import module java.sql;
```

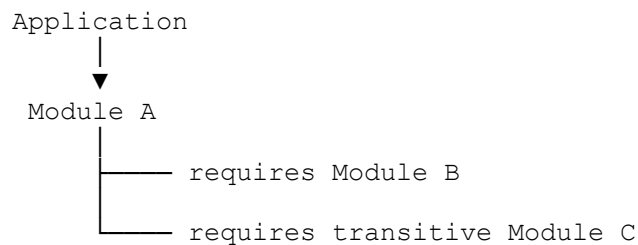
is **module-level**.

This distinction is central to understanding JEP 476.

10. Module imports and transitive dependencies

A module can depend on other modules.

For example:



Module import resolution takes the module graph and accessibility rules into account.

Therefore, import module is more sophisticated than simply importing every package from a single module.

This is one reason the feature is useful in **modular applications**, but also one reason developers should understand JPMS accessibility rules.

11. Name conflicts

One disadvantage is that importing many types can introduce conflicts.

Suppose two accessible APIs contain types with the same simple name:

```
Module A → Result  
Module B → Result
```

If your code imports both modules:

```
import module moduleA;  
import module moduleB;
```

then:

Result result;

may become ambiguous.

In such cases, an explicit import or fully qualified name can resolve the conflict.

For example:

moduleA.Result result;

or an explicit type import where appropriate.

Practical rule

Use module imports when they make the code simpler; use explicit imports when name resolution becomes ambiguous.

12. Module Import vs traditional imports

Aspect	Module Import	Explicit Import	Package Wildcard
Boilerplate	★★★★★	★★	★★★★
Explicit dependency visibility	★★★	★★★★★	★★
Large API convenience	★★★★★	★★	★★★★
Name-conflict risk	⚠ Higher	⚠ Low	⚠ Moderate
Module-system alignment	★★★★★	★★★	★★
Simplicity	★★★★	★★★	★★★★
Fine-grained control	★★	★★★★★	★★★
Best for public APIs	★★★	★★★★★	★★★
Best for prototyping	★★★★★	★★★	★★★★
Java 23 preview requirement	✓	✗	✗

--	--	--	--

13. When NOT to use it

Situation	Better choice	Reason
Only one type is needed	Explicit import	More precise.
Only two types are needed	Explicit imports	Minimal overhead.
Multiple modules contain similarly named types	Explicit imports	Avoid ambiguity.
Highly strict coding standards	Explicit imports	Dependencies are immediately visible.
Public library where API clarity is critical	Explicit imports	Easier to see exactly what is referenced.
Project does not use JPMS	Traditional imports	Module import adds little value.
Java 22 or older target	Package/type imports	Feature isn't available as a standard Java 23 feature.
Preview features prohibited	Traditional imports	No preview dependency.
Very small class	Traditional imports	Module import may be overkill.

14. Alternate features

Requirement	Alternative	When to use
Import one class	Single-type import	Most precise choice.
Import many classes from one package	Package wildcard (*)	Convenient for package-heavy code.
Import static methods/constants	Static import	For utility methods/constants.
Manage module	module-info.java	Defines actual module

dependencies		dependencies.
Discover available APIs	IDE autocomplete / Javadoc	Better for API exploration.
Avoid imports entirely	Fully qualified names	Useful for occasional conflicting types.
Legacy application	Classpath + normal imports	When JPMS isn't being used.
API organization	Explicit imports + package structure	Better for highly controlled source code.

15. import module vs module-info.java

These two concepts should **not** be confused.

module-info.java

Defines dependencies:

```
module com.example.application {
    requires java.sql;
}
```

Source file

Uses the module's types:

```
import module java.sql;
```

So:

```

module-info.java
    |
    | defines module dependency
    v
requires java.sql
    |
    v
Java source file
    |
    | imports accessible types
    v
import module java.sql
```

Feature	Purpose
requires java.sql;	Module dependency declaration
import module java.sql;	Source-code type import

import module does **not** replace requires.

16. Recommended usage strategy

Development stage	Recommendation
Learning Java APIs	👉 Module imports can be convenient
Prototyping	👉 Good
Small examples	👉 Good
Large modular application	👉 Consider it
Internal application	👉 Evaluate based on team style
Public library	👉 Prefer explicit imports where clarity matters
Highly regulated codebase	👉 Follow project coding standards
Conflicting type names	👉 Avoid module import
Java 23 preview experimentation	👉 Good
Long-term stable Java 23 production	👉 Consider preview-feature policy carefully

17. Performance impact

Module Import Declarations are a **source-code convenience feature**.

They do **not** make application execution faster.

Area	Performance impact
------	--------------------

Runtime execution	☐ None
CPU performance	☐ None
Memory usage	☐ No meaningful runtime improvement
Startup time	☐ No meaningful benefit
Compilation	☒ Compiler must resolve module imports
Developer productivity	☒ Potentially improved
Source-code maintainability	☒ Potentially improved

Therefore:

Do not adopt Module Import Declarations as a Java performance optimization.

18. Advantages vs disadvantages — final assessment

Category	Assessment
Reducing import boilerplate	☒ Excellent
API exploration	☒ Excellent
Educational code	☒ Excellent
Large module APIs	☒ Excellent
Modular applications	☒ Very useful
Explicit dependency visibility	☒ Moderate
Name-conflict avoidance	☒ Worse than explicit imports
Simple classes	☒ Limited benefit
Legacy applications	☒ Limited benefit
Runtime performance	☐ No effect
Learning curve	☒ Requires JPMS understanding
Java 23 production stability	☒ Preview feature

Public-library source clarity	🔗 Depends on project style

19. Final recommendation

Question	Answer
Should I learn it?	✓ Yes, particularly if you're learning Java's module system and modern Java syntax.
Should I use it everywhere?	✗ No.
Best use case	Using many types from a large module , especially during prototyping, examples, and modular applications.
Best alternative	Explicit imports when source-level precision is more important.
Does it replace module-info.java?	✗ No. module-info.java defines dependencies; import module imports types for source code.
Does it replace import java.sql.*?	✗ No. A package wildcard and module import operate at different levels.
Does it improve runtime performance?	✗ No.
Biggest advantage	Less import boilerplate when using many APIs from a module.
Biggest disadvantage	Reduced precision and potentially more name conflicts , plus preview-feature considerations in Java 23.
Best production approach	Use it selectively where it genuinely improves readability; retain explicit imports where precise dependencies or name clarity matter.

One-line takeaway

Module Import Declarations bring imports up from the package level to the module level: `import module java.sql`; can make an entire module's accessible exported types available, making large APIs easier to use while trading away some import-level precision.

Java 23 — Implicitly Declared Classes and Instance main Methods (Third Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Implicitly Declared Classes and Instance main Methods
UniqueClassName	ImplicitClassesInstanceMain
JEP	JEP 477
Status	Third Preview
First introduced	Java 21 as JEP 445
Primary goal	Make small Java programs easier to write and teach
Main benefit	Removes unnecessary class, public, static, and String[] args ceremony for simple programs
Related feature	JEP 476 — Module Import Declarations
Performance impact	Primarily a syntax/learning improvement , not a direct runtime-performance improvement

Java 23 re-previewed this feature as JEP 477. In Java 23, implicitly declared classes also receive automatic access to static methods from `java.io.IO` and public top-level classes/interfaces exported by `java.base`. ([Oracle Docs](#))

1. What problem does this feature solve?

Traditional Java requires even the smallest program to introduce several concepts:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

A beginner has to understand:

- class
- public
- static
- void
- String[]
- main()

before simply printing "Hello World".

Java 23 allows a much simpler form:

```
void main() {  
    System.out.println("Hello World");  
}
```

The source file is treated as a **simple compilation unit**, and Java implicitly creates a top-level class around the declarations. ([Oracle Docs](#))

2. How it works

Traditional Java	Java 23 Preview
Explicit class required	Class can be implicit
public static void main(String[] args)	void main() can be used
public commonly used	Not required for the entry point
static required for traditional main	Instance main allowed
String[] args required in traditional form	Parameter can be omitted
Explicit imports often required	java.base public types automatically available
System.out.println()	System.out.println() or convenient IO.println()
More ceremony	Less ceremony

An implicitly declared class is a top-level, final class in the unnamed package with Object as its superclass. It receives an implicitly declared no-argument constructor. ([Oracle Docs](#))

3. Basic example

Traditional Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Java 23 Preview

```
void main() {  
    System.out.println("Hello World");  
}
```

Or, using the Java 23 simple console I/O support:

```
void main() {  
    IO.println("Hello World");  
}
```

The Java 23 feature automatically makes the static methods of `java.io.IO` available to simple compilation units. ([Oracle Docs](#))

4. Advantages

#	Advantage	Explanation	Example / Impact
1	Much less boilerplate	Removes unnecessary class and main ceremony	<code>void main() { ... }</code>
2	Beginner friendly	Students can learn variables and control flow before classes	Excellent for introductory programming
3	Instance <code>main()</code> supported	<code>main()</code> doesn't have to be static	<code>void main() { ... }</code>
4	No mandatory <code>String[] args</code>	Programs that don't need command-line arguments can omit them	<code>void main()</code>
5	Simple source files	A .java file can behave almost like a small script	Useful for experiments

6	Smooth learning progression	Beginners can later add an explicit class	Small program → normal Java application
7	Simple console I/O	Java 23 provides automatic access to java.io.IO static methods	println("Hello")
8	Less cognitive load	Beginners focus on programming concepts rather than Java scaffolding	Teaching benefit
9	Useful for small utilities	Tiny programs can be concise	Data conversion, calculations
10	Still Java	It isn't a separate scripting language	Same Java language/JVM ecosystem
11	Rapid experimentation	Small ideas can be tested quickly	Algorithm experiments
12	Good for classroom exercises	Students can concentrate on the problem itself	Loops, methods, arrays, conditions

Oracle specifically describes the feature as improving Java's support for "**programming-in-the-small**", while allowing programs to later grow into normal Java programs. ([Oracle Docs](#))

5. Disadvantages

#	Disadvantage	Explanation	Concern
1	Preview feature	Java 23 implementation is not permanent	Syntax/specification could change
2	Requires preview options	Compilation/execution requires preview support	Not ideal for production
3	Less familiar to experienced Java developers	Most Java code uses explicit classes	Can initially look unusual
4	Limited scalability	Implicit classes are intended for simple programs	Poor fit for large applications

5	Unnamed package	Implicit classes reside in the unnamed package	Not suitable for enterprise namespace organization
6	Cannot be referenced by name	The implicit class cannot be directly referenced from Java source	Limits reuse
7	No normal API-oriented design	The construct is intended primarily for standalone programs	Not appropriate for library APIs
8	Potential educational confusion	Students eventually need to understand normal classes anyway	Transition must be explained
9	Tooling/documentation limitations	Some conventional tooling assumptions are based on explicit classes	May be less convenient
10	Not a performance feature	Removing syntax does not automatically make the program faster	No inherent throughput improvement
11	Production convention mismatch	Teams generally expect explicit packages/classes	Codebase consistency issue
12	Migration considerations	Code using preview syntax may require changes in future releases	Long-term compatibility concern

Oracle explicitly identifies this as a **preview feature**, meaning its design is not permanent and it may change or even disappear in a future Java release. ([Oracle Docs](#))

6. When should you use it?

Situation	Use?	Reason	Example
Learning Java	✓ Yes	Reduces unnecessary syntax	<code>void main() { ... }</code>
Teaching beginners	✓ Yes	Focuses on programming fundamentals	Loops/variables
Small one-file	✓ Yes	Minimal ceremony	Calculator

programs			
Algorithm experiments	✓ Yes	Quickly test an idea	Sorting algorithm
Coding demonstrations	✓ Yes	Short examples	Java tutorials
Temporary utilities	✓ Yes	Convenient for small tasks	File/data conversion
REPL-like experimentation	✓ Yes	Faster experimentation	Small calculations
Small command-line utility	? Maybe	Good if it remains simple	Log analyzer
Large application	✗ No	Explicit architecture is better	Spring application
Enterprise system	✗ No	Packages/classes/modules are valuable	Banking system
Public library	✗ No	Explicit API classes are required	Java SDK
Multi-team application	✗ No	Clear namespaces are important	Microservices
Long-lived production code	? Usually no	Preview status is a concern	Core business system

7. When should you NOT use it?

Scenario	Why avoid it?	Better choice
Large enterprise application	Needs explicit organization	Normal classes/packages
Public API/library	Implicit class isn't designed as a reusable API	Explicit public classes
Framework development	Frameworks generally rely on explicit types and metadata	Explicit classes
Multi-module application	Implicit classes belong to the unnamed package	Named packages/modules

Large team	Explicit structure improves maintainability	Standard Java structure
Production system requiring long-term stability	Preview feature risk	Standard Java
Complex domain model	Many classes/interfaces are needed	Explicit classes/records/interfaces
Application with inheritance	Implicit class has restricted structure	Explicit class hierarchy
Class-based dependency injection	Explicit types are preferable	Normal classes
Stable coding standard	Preview syntax may not fit team standards	Conventional Java

8. Real-time use cases

Use Case 1 — Teaching Java

Traditional

```
public class Sum {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;

        System.out.println(a + b);
    }
}
```

Java 23 Preview

```
void main() {
    int a = 10;
    int b = 20;

    IO.println(a + b);
}
```

Benefit

Students immediately see:

variable → calculation → output

instead of first learning:

class → public → static → void → String[] → main

This is one of the feature's primary design goals. ([Oracle Docs](#))

9. Real-time use case — Small calculator

```
void main() {  
    int price = 1000;  
    int quantity = 5;  
  
    int total = price * quantity;  
  
    IO.println("Total = " + total);  
}
```

Suitable for: learning exercises, demonstrations, temporary utilities.

Not suitable for: a production billing system.

10. Real-time use case — Small data conversion utility

```
void main() {  
    double celsius = 25.0;  
  
    double fahrenheit = (celsius * 9 / 5) + 32;  
  
    IO.println("Fahrenheit = " + fahrenheit);  
}
```

This is an excellent example of **programming-in-the-small**.

11. Real-time use case — Instance main()

One of the important aspects is that main() can be an **instance method**.

```
void main() {
```

```
String application = "OrderProcessor";

IO.println("Starting " + application);
}
```

Conceptually, the JVM can create an instance of the initial class and invoke the instance `main()` method. The Java specification defines candidate main methods with either a single parameter or no parameters, and allows the selected method to be static or instance-based. ([Oracle Docs](#))

12. Real-time use case — Helper methods

The feature isn't limited to one statement.

```
int square(int value) {
    return value * value;
}

void main() {
    int result = square(10);

    IO.println("Result = " + result);
}
```

This is useful for small algorithm demonstrations.

13. Real-time use case — Fields

You can also declare fields in the simple compilation unit.

```
String applicationName = "Demo Application";

void main() {
    IO.println(applicationName);
}
```

The declarations are treated as members of the implicitly declared class. ([Oracle Docs](#))

14. Growing from beginner code to normal Java

This is one of the most interesting design goals.

Stage 1 — Very simple

```
void main() {  
    IO.println("Hello");  
}
```

Stage 2 — Add methods

```
String message() {  
    return "Hello";  
}
```

```
void main() {  
    IO.println(message());  
}
```

Stage 3 — Move to explicit class

```
public class Hello {  
    String message() {  
        return "Hello";  
    }  
  
    public static void main(String[] args) {  
        Hello app = new Hello();  
        System.out.println(app.message());  
    }  
}
```

So the feature provides a potential learning path:

```
Simple program  
↓  
Methods / variables  
↓  
Classes  
↓  
Objects  
↓  
Packages  
↓  
Modules  
↓  
Large application
```

This gradual-growth approach is explicitly part of the feature's motivation. ([Oracle Docs](#))

15. Important Java 23 improvement

Compared with earlier previews, Java 23 introduced automatic imports for simple compilation units.

Java 23 capability	Explanation
java.io.IO	Static methods for simple textual console I/O are automatically available
java.base exported packages	Public top-level classes/interfaces can be used by simple name
java.lang	Normal Java implicit imports continue to apply
Module Import Declarations	JEP 477 works together with JEP 476

For example, simple source code can use types from java.base without writing the corresponding import in many cases. ([Oracle Docs](#))

16. Comparison with traditional Java

Feature	Traditional Java	Implicit Class + Instance main
Explicit class	Required	Optional
public class	Commonly required	Not required
static main	Required for traditional entry point	Not required
String[] args	Required in traditional form	Optional
Instance main	✗	✓
Named package	✓	✗ — unnamed package
Reusable API	✓	Not the intended use
Large applications	✓	✗
Small programs	More verbose	Very concise
Beginner learning	More difficult initially	Easier

Production stability	Stable language constructs	Preview in Java 23
Performance	Normal	Essentially same execution model; syntax simplification is the goal

17. Alternate features / alternatives

There isn't one exact replacement because this feature addresses **program structure and entry-point syntax**, rather than a specific runtime capability.

Alternative	What it provides	When preferable	Compared with JEP 477
Traditional public static void main	Standard Java entry point	Production applications	Most compatible
Explicit class + instance main	Object-oriented entry point	When you want instance state	More explicit
JShell	Interactive Java experimentation	REPL-style experimentation	Better for interactive exploration
Single-file Java source launch	Run .java directly	Small utilities/scripts	Doesn't require implicit class syntax
Kotlin scripting	Script-like JVM programming	Script-heavy applications	More script-oriented
Groovy	Dynamic JVM scripting	Automation/scripts	More dynamic
Python	Very concise scripting	General-purpose scripting	Not Java/JVM
Java records	Concise data classes	Data modeling	Solves a different problem
Explicit classes + packages	Structured applications	Enterprise software	Better for large systems

18. JEP 477 vs JShell

Aspect	JEP 477	JShell
---------------	----------------	---------------

Purpose	Small Java programs	Interactive experimentation
Source file	.java	JShell session
Compilable application	✓	Not primarily
IDE integration	✓	Limited/different workflow
Production deployment	Possible technically, but preview concern in Java 23	Not intended
Beginner friendly	✓	✓
Persistent source	✓	Not normally
Best for	Small programs	Experiments/learning

Rule of thumb:

Use JShell for exploration; use JEP 477-style source files for small standalone programs.

19. JEP 477 vs traditional main

Requirement	Traditional	JEP 477
Explicit class	✓	✗
public	Usually	✗
static	✓	✗ for instance main
String[] args	Usually	Optional
Named package	Possible	✗
Large application support	✓	Limited
Beginner simplicity	☆☆	☆☆☆☆☆
Enterprise suitability	☆☆☆☆☆	☆☆
API/library suitability	☆☆☆☆☆	☆
Small utility suitability	☆☆☆	☆☆☆☆☆

20. Impact on performance

This feature should **not** be considered a performance optimization.

Area	Impact
Startup time	Generally not the purpose of the feature
CPU performance	No inherent improvement
Memory consumption	No meaningful benefit expected merely from syntax
Throughput	No direct improvement
Code size	Source code becomes smaller
Developer productivity	Potentially significant improvement for small programs
Learning curve	Significant improvement for beginners
Maintainability	Better for tiny programs; worse if misused for large systems

The primary objective is reducing ceremony and supporting programming-in-the-small, not improving JVM execution performance. ([Oracle Docs](#))

21. Production recommendation

Project type	Recommendation
Java classroom exercises	🔍 Use
Beginner tutorials	🔍 Use
Algorithm demonstrations	🔍 Use
Small experiments	🔍 Use
Temporary one-file utilities	🔍 Consider
Small personal tools	🔍 Consider

Production CLI utility	❓ Prefer normal Java unless preview is acceptable
Enterprise application	❓ Don't use
Spring Boot application	❓ Don't use as the primary structure
Public Java library	❓ Don't use
Large multi-module system	❓ Don't use
Long-lived production application	❓ Prefer standard Java

22. Final summary

Aspect	Verdict
Main purpose	Simplify small Java programs
Best benefit	Removes class/static/String[] args ceremony
Best audience	Beginners, educators, small-program developers
Best use case	Small standalone Java programs
Biggest disadvantage	Preview status + poor fit for large structured applications
Runtime performance	No significant inherent improvement
Enterprise suitability	Low
Educational suitability	Excellent
Alternative for production	Traditional explicit classes and main
Alternative for interactive experimentation	JShell
Java 23 significance	Third preview with improved simple I/O and automatic java.base imports

Bottom line

JEP 477 is primarily a developer/education productivity feature, not a performance feature. It makes Java programs look much closer to a simple script while preserving the path to normal Java programming. For **small programs, teaching, experiments, and demonstrations**, it is highly useful; for **enterprise applications, reusable libraries, and long-lived production systems**, traditional explicit classes, packages, and modules remain the better choice. ([Oracle Docs](#))

Java 23 — Structured Concurrency (Third Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Structured Concurrency
UniqueClassName	StructuredConcurrency
JEP	JEP 480
Status	Third Preview
First introduced	Java 19 as a Preview
Package	java.util.concurrent
Primary API	StructuredTaskScope
Main purpose	Treat related concurrent tasks as a single unit of work
Works especially well with	Virtual threads
Core principle	Child tasks should have a clear lifetime bounded by their parent task
Main benefit	Easier cancellation, error handling, task lifetime management and observability
Performance goal	Better management of concurrency rather than automatically making individual operations faster

Structured Concurrency was introduced to make concurrent code easier to reason about by treating groups of related tasks as a single structured operation. In Java 23, JEP 480 was its **third preview**.

1. What is Structured Concurrency?

Traditional concurrent Java code often looks like this:

```
ExecutorService executor = Executors.newFixedThreadPool(10);
```

```
Future<String> user =  
    executor.submit(() -> getUser());
```

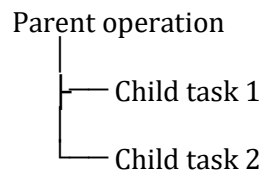
```
Future<String> orders =  
    executor.submit(() -> getOrders());
```

```
String u = user.get();  
String o = orders.get();
```

There are several problems:

- Who owns these tasks?
- What happens if one fails?
- Should the other task be cancelled?
- What happens if the parent operation is cancelled?
- Who waits for the tasks?
- What happens if a task continues running after the request has finished?

Structured concurrency tries to make the relationship explicit:



Parent cannot finish
until its child tasks are handled.

This is similar to structured programming:

if / for / while

give structure to sequential execution.

Structured concurrency gives structure to:

concurrent execution

2. Basic Example

A simplified Java 23 example:

```
import java.util.concurrent.StructuredTaskScope;

String getUserProfile() throws Exception {
    try (var scope = StructuredTaskScope.open()) {

        var userTask = scope.fork(this::getUser);
        var orderTask = scope.fork(this::getOrders);

        scope.join();

        return userTask.get() + " : " + orderTask.get();
    }
}
```

Conceptually:

```
getUserProfile()
├── StructuredTaskScope
│   ├── getUser()
│   └── getOrders()
```

The scope establishes a clear boundary around the child tasks.

Note: Structured Concurrency was still a preview API in Java 23, so preview compilation/execution options are required.

3. Why is this useful?

Imagine an e-commerce request:

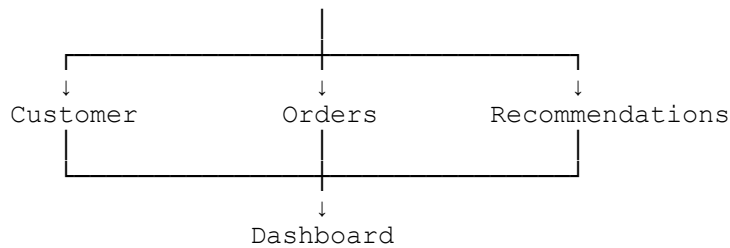
GET /customer/123/dashboard

The server needs:

Customer
Orders
Recommendations
Notifications

These operations can execute concurrently:

Dashboard Request



If the dashboard request is cancelled, the related child tasks should normally be cancelled too.

Structured concurrency provides a natural mechanism for managing this relationship.

4. Advantages

#	Advantage	Explanation	Real-world benefit
1	Clear task lifetime	Child tasks live within a defined scope	Prevents orphan tasks
2	Better cancellation	Related tasks can be cancelled together	Saves resources
3	Simpler error handling	Task failures can be handled at the scope level	Easier debugging
4	Parent-child relationship	Makes concurrency hierarchy explicit	Easier reasoning
5	Works naturally with virtual threads	Lightweight concurrent tasks fit well with virtual threads	High concurrency
6	Improved observability	Task relationships are easier to understand	Better monitoring
7	Avoids task leaks	Scope boundaries help prevent forgotten tasks	More reliable servers
8	Simplifies concurrent code	Reduces scattered Future management	Cleaner code
9	Supports fail-fast designs	Can stop related work after an important failure	Lower latency/resource usage
10	Better request-scoped	Tasks can map naturally to a request	Excellent for web services

	concurrency		
11	Structured cancellation	Cancellation propagates through related tasks	Better shutdown behavior
12	Easier reasoning	Concurrency becomes hierarchical	Lower cognitive complexity
13	Resource ownership becomes clearer	Scope owns child tasks	Easier lifecycle management
14	Composability	Structured concurrent operations can be combined	Useful in layered applications

5. Disadvantages

#	Disadvantage	Explanation	Concern
1	Preview feature in Java 23	API was not yet final	Production compatibility concern
2	Learning curve	Developers must understand structured concurrency concepts	Training required
3	Not suitable for every asynchronous workflow	Some tasks intentionally outlive their parent	Structured scope may be inappropriate
4	API evolution	Preview APIs may change	Migration effort
5	Requires careful cancellation design	Incorrect cancellation handling can cause problems	Need good architecture
6	Doesn't eliminate concurrency complexity	It organizes concurrency but doesn't make it trivial	Developers still need concurrency knowledge
7	Potentially unnecessary for simple sequential code	Adds concepts where concurrency isn't needed	Overengineering
8	Long-running	Their lifetime may	Use another mechanism

	background jobs don't fit naturally	intentionally exceed the request	
9	Debugging still requires concurrency knowledge	Race conditions and shared state can remain	Not a magic solution
10	Integration considerations	Existing executor/Future-based systems may need adaptation	Migration effort
11	Preview flags	Applications must enable preview functionality	Deployment complexity
12	Cancellation semantics require discipline	Tasks must respond properly to interruption/cancellation	Poorly behaved tasks can still cause problems

6. When should you use Structured Concurrency?

Situation	Use?	Why
Multiple independent API calls	✓ Yes	Calls can run concurrently
Web request aggregation	✓ Yes	Natural parent-child relationship
Microservice aggregation	✓ Yes	Parallel downstream calls
Database + external API calls	✓ Yes	Reduce overall latency
Parallel data retrieval	✓ Yes	Related operations share a lifetime
"First successful result"	✓ Yes	Can cancel unnecessary tasks
Request cancellation	✓ Yes	Child work can follow parent cancellation
Virtual-thread applications	✓ Yes	Strong combination
Parallel validation	✓ Yes	Related tasks can be grouped
Simple sequential	✗ No	No concurrency required

algorithm		
Independent background job	✗ Usually no	Job may outlive request
Message consumer with independent tasks	? Maybe	Depends on task lifetime
Fire-and-forget notification	✗ Usually no	Intentionally independent lifetime
Scheduled recurring task	✗ Usually no	Scheduler owns lifetime
Batch processing	? Depends	Useful when tasks have a common scope

7. When should you NOT use it?

Scenario	Why not?	Alternative
Fire-and-forget work	Task intentionally outlives parent	Executor/service/queue
Scheduled jobs	Lifetime isn't tied to caller	ScheduledExecutorService
Message queues	Consumer controls task lifetime	Messaging framework
Independent background processing	No parent-child relationship	Executor
Simple single-threaded computation	Concurrency provides no benefit	Normal method call
Existing mature async framework	Migration may provide little value	Existing framework
Long-lived workers	Structured scope isn't their natural lifetime	Executor/service
Complex workflow orchestration	May need durable workflow engine	Workflow platform
Tasks intentionally survive request	Parent-child cancellation is undesirable	Explicit background executor

cancellation		
--------------	--	--

8. Real-time Use Case — Microservice API Aggregation

Suppose an e-commerce application needs:

Customer Service
Order Service
Recommendation Service

Traditional sequential approach

```
Customer customer = customerService.getCustomer(id);
Orders orders = orderService.getOrders(id);
Recommendations recommendations =
    recommendationService.getRecommendations(id);
```

Approximate latency:

```
100 ms + 150 ms + 200 ms
= 450 ms
```

Concurrent approach

```
Customer      _____ 100 ms
Orders        _____ 150 ms
Recommendations _____ 200 ms
```

Total approximately:

```
max(100, 150, 200)
≈ 200 ms
```

This is an excellent use case for structured concurrency because all three tasks belong to the same dashboard request.

9. Example — Parallel service calls

Conceptually:

```
Response getDashboard(long customerId) throws Exception {
    try (var scope = StructuredTaskScope.open()) {
```

```

var customer = scope.fork(
  () -> customerService.getCustomer(customerId)
);

var orders = scope.fork(
  () -> orderService.getOrders(customerId)
);

var recommendations = scope.fork(
  () -> recommendationService.getRecommendations(customerId)
);

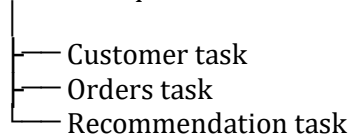
scope.join();

return new Response(
  customer.get(),
  orders.get(),
  recommendations.get()
);
}
}

```

The important architectural relationship is:

Dashboard request



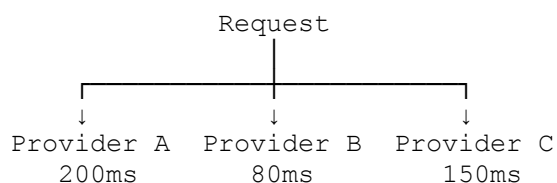
When the dashboard request ends, the scope also ends.

10. Real-time Use Case — First Successful Result

Suppose an application has multiple providers:

Provider A
 Provider B
 Provider C

You want the fastest successful response.



↓
First success

After Provider B succeeds, continuing the other operations may be unnecessary.

Structured concurrency is well suited to this type of **race/fail-fast** pattern.

11. Real-time Use Case — Parallel Validation

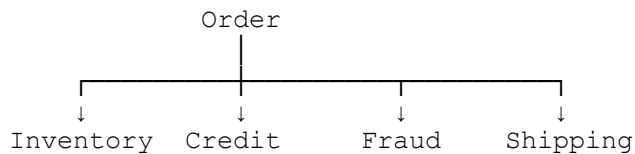
Suppose an order needs:

Inventory validation
Credit validation
Fraud validation
Shipping validation

Instead of:

Inventory → Credit → Fraud → Shipping

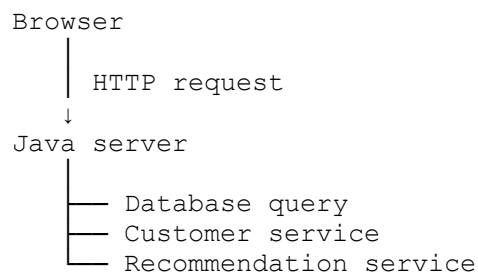
they can potentially execute as:



If a critical validation fails, related tasks can be cancelled depending on the chosen scope policy.

12. Real-time Use Case — Web Request Cancellation

Consider:



The user closes the browser.

Without structured lifecycle management, downstream work might continue unnecessarily.

With structured concurrency:

HTTP Request cancelled



Parent scope cancelled



Child tasks cancelled



Resources released

This is particularly valuable in high-load services.

13. Structured Concurrency + Virtual Threads

These features complement each other.

Feature	Main purpose
Virtual Threads	Make large numbers of concurrent tasks inexpensive
Structured Concurrency	Organize and manage those concurrent tasks
Together	Lightweight + well-managed concurrency

Think of them as:

Virtual Threads



"How can I run many tasks?"

Structured Concurrency



"How should I organize and control those tasks?"

A modern Java server can therefore use:

HTTP request



StructuredTaskScope



- └ Virtual Thread → DB call
- └ Virtual Thread → REST call
- └ Virtual Thread → cache call

14. Structured Concurrency vs ExecutorService

Aspect	ExecutorService	Structured Concurrency
Task submission	submit()	fork()
Task lifetime	Often manually managed	Scope-based
Parent-child relationship	Implicit	Explicit
Cancellation	Manual	Scope-oriented
Error handling	Manual	Structured
Task ownership	Can be unclear	Scope owns subtasks
Future management	Required	Simplified
Request-scoped work	Possible	Excellent fit
Fire-and-forget	Good	Poor fit
Long-lived background tasks	Good	Poor fit
Preview in Java 23	No	Yes
Best use	General task execution	Related concurrent subtasks

15. Structured Concurrency vs CompletableFuture

Aspect	CompletableFuture	Structured Concurrency
Programming model	Future/pipeline based	Scope based
Composition	Excellent	Excellent for task groups
Cancellation	Can become complicated	More structured

Error propagation	Explicit composition	Scope-oriented
Task lifetime	Can escape caller	Bounded by scope
Async pipelines	Excellent	Not its primary purpose
Request aggregation	Good	Excellent
Fire-and-forget	Possible	Not ideal
Virtual threads	Can be used	Designed to complement them
Readability	Can become complex with chains	Often simpler for task groups
Preview status in Java 23	Final API	Third Preview

16. Alternate features

If Structured Concurrency is unavailable or inappropriate, choose according to the problem.

Requirement	Preferred alternative
General concurrent task execution	ExecutorService
Async composition	CompletableFuture
Lightweight concurrent blocking tasks	Virtual Threads
Interactive async reactive pipelines	Reactive Streams / Reactor
Long-running background jobs	Executor + application-managed lifecycle
Scheduled tasks	ScheduledExecutorService
Distributed asynchronous processing	Message queue
Durable workflows	Workflow engine
Parallel CPU computation	Fork/Join Framework
Simple sequential work	Normal method calls

17. Most important alternate: Virtual Threads

If the question is:

"What is the closest Java feature I can use instead?"

The answer depends on what you're replacing.

If your objective is...	Use
Run many blocking tasks efficiently	Virtual Threads
Organize related concurrent tasks	Structured Concurrency
Compose asynchronous results	CompletableFuture
Execute tasks from a pool	ExecutorService
CPU parallelism	Fork/Join
Reactive event pipelines	Reactive Streams

So:

Virtual Threads are not a direct replacement for Structured Concurrency.

They solve different problems.

Virtual Threads
= execution mechanism

Structured Concurrency
= concurrency organization/lifecycle mechanism

18. Structured Concurrency vs Parallel Streams

Aspect	Structured Concurrency	Parallel Streams
Main purpose	Concurrent tasks	Data parallelism
API	StructuredTaskScope	Stream.parallel()
Multiple independent services	✓	✗

Collection processing	Possible but not primary	✓ Excellent
Cancellation control	Stronger	Limited
Task lifecycle	Explicit scope	Stream operation
I/O-bound work	✓ Good	Usually not ideal
CPU-bound data processing	?	✓ Good
Web request aggregation	Excellent	Poor
Virtual threads	Works naturally	Uses common pool by default
Preview in Java 23	Yes	Final

19. Structured Concurrency vs Reactive Programming

Aspect	Structured Concurrency	Reactive programming
Programming style	Direct/blocking style	Non-blocking/reactive
Virtual threads	Excellent fit	Not required
Learning curve	Moderate	Higher
Debugging	Generally easier	Often more complex
Request aggregation	Excellent	Excellent
Backpressure	Not its primary purpose	Strong
Streaming events	Not primary	Excellent
High-volume event pipelines	?	Excellent
Traditional REST application	Excellent	Good
Existing reactive architecture	May not be worth migration	Keep reactive

20. Performance considerations

Structured concurrency itself should **not** be viewed as a magic performance optimization.

The major performance opportunity comes from **parallelizing independent operations**.

For example:

Sequential:

A = 100 ms

B = 150 ms

C = 200 ms

Total $\approx$ 450 ms

With concurrency:

A ——— 100 ms

B ——— 150 ms

C ——— 200 ms

Total $\approx$ 200 ms

But this only works when:

1. Tasks are independent.
2. External services can handle concurrent requests.
3. Database connections aren't the bottleneck.
4. Network latency dominates enough to benefit.
5. The application has sufficient resources.
6. Parallel execution doesn't create excessive contention.

Therefore:

Structured concurrency can reduce end-to-end latency by enabling safe concurrency, but it does not automatically make every program faster.

21. Common mistakes

Mistake	Problem	Better approach
Using it for every method	Unnecessary complexity	Use it for related concurrent tasks
Parallelizing dependent tasks	Tasks can't actually execute independently	Keep dependency chain sequential

Ignoring cancellation	Resources may remain occupied	Design cancellation properly
Running too many external calls	Can overload dependencies	Apply limits/backpressure
Treating it as a replacement for queues	Wrong lifecycle model	Use messaging for durable work
Using it for scheduled jobs	Parent-child model doesn't fit	Scheduler
Assuming it improves CPU performance	Not necessarily true	Benchmark
Ignoring downstream limits	Database/API may become bottleneck	Control concurrency
Using preview API in critical production	Compatibility risk	Prefer finalized APIs where possible

22. Decision table

Requirement	Structured Concurrency?	Recommended approach
Three REST calls needed for one HTTP response	☑ Yes	Structured Concurrency
Database + cache + service lookup	☑ Yes	Structured Concurrency
First successful response among providers	☑ Yes	Structured Concurrency
Parallel validation	☑ Yes	Structured Concurrency
Request cancellation propagation	☑ Yes	Structured Concurrency
Thousands of blocking I/O operations	☑ Yes + Virtual Threads	Strong combination
Process 10 million array elements	☑	Vector API / parallel processing

Fire-and-forget email	?	Queue/executor
Scheduled report	?	Scheduler
Durable background processing	?	Message queue/workflow
Reactive event pipeline	?	Reactive Streams
Simple sequential calculation	?	Normal Java
CPU-intensive recursive computation	?	Fork/Join

23. Final recommendation

Category	Recommendation
Learning concurrency	? Learn it
Modern Java server applications	? Very valuable concept
Virtual-thread applications	? Excellent combination
Microservice aggregation	? Excellent
Parallel independent I/O	? Excellent
Request cancellation	? Excellent
CPU-intensive parallel computation	? Consider Fork/Join/Vector API
Background jobs	? Usually avoid
Scheduled jobs	? Usually avoid
Message-driven durable work	? Usually avoid
Simple sequential code	? Avoid
Java 23 production application	? Caution because it was a preview
Long-term API commitment in Java 23	? Prefer finalized APIs

One-line takeaway

Structured Concurrency (Third Preview) is about making concurrent tasks have a clear parent, lifetime, cancellation, and error-handling boundary—especially powerful for request-scoped I/O workloads and virtual-thread-based applications, but unnecessary for independent background work or simple sequential programs.

Java 23 — Scoped Values (Third Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Scoped Values
UniqueClassName	ScopedValues
JEP	JEP 481
Status	Third Preview
First introduced	Java 20 as an Incubator
Package	java.lang
Main API	ScopedValue<T>
Primary purpose	Safely and efficiently share immutable data across a call chain without passing it through every method parameter
Main alternative	ThreadLocal<T>
Best companion feature	Structured Concurrency
Especially useful with	Virtual Threads
Main characteristic	One-way, bounded-lifetime data sharing
Runtime-performance goal	Efficient context sharing, particularly with large numbers of virtual threads

Oracle describes a scoped value as a value that can be safely and efficiently shared with methods without using method parameters. Java 23's version was the **third preview** of JEP 481. ([Oracle Docs](#))

1. What problem does Scoped Values solve?

Normally, if some piece of context is needed deep inside a call chain, you have to pass it through every method:

```
void processRequest(RequestContext context) {
    service(context);
}

void service(RequestContext context) {
    repository(context);
}

void repository(RequestContext context) {
    log(context);
}

void log(RequestContext context) {
    System.out.println(context.requestId());
}
```

The problem is that context may be relevant only to `log()`, but every intermediate method must carry it.

With a ScopedValue:

```
processRequest()
├── service()
│   ├── repository()
│   │   └── log()
│   │       ↓
│   │   ScopedValue.get()
```

The context is available throughout the **dynamic scope** without being explicitly passed through every method. The Java API describes this as effectively acting like an **implicit method parameter**. ([Oracle Docs](#))

2. Basic example

A simplified Java 23 example:

```
static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

void main() {
    ScopedValue.runWhere(
        REQUEST_ID,
        "REQ-1001",
        () -> processRequest()
    );
}

static void processRequest() {
    service();
}

static void service() {
    repository();
}

static void repository() {
    IO.println("Request ID = " + REQUEST_ID.get());
}
```

Conceptually:

```
runWhere()
  |
  | binds REQUEST_ID = "REQ-1001"
  ↓
processRequest()
  ↓
service()
  ↓
repository()
  ↓
REQUEST_ID.get()
  ↓
"REQ-1001"
```

When the operation completes, the binding automatically goes away. It does not remain attached to the thread indefinitely. ([Oracle Docs](#))

3. Scoped Value vs normal method parameter

Aspect	Method Parameter	Scoped Value
--------	------------------	--------------

Data passed explicitly	✓	✗
Type-safe	✓	✓
Visible in method signature	✓	✗
Can be accessed by deep callees	Only if passed	✓
Intermediate methods need parameter	✓	✗
Lifetime	Method/call dependent	Explicit dynamic scope
Best for	Normal business data	Cross-cutting context
Hidden dependency	✗	☒ Yes
Good for request context	☒	✓ Excellent
Good for ordinary business values	✓	✗ Usually unnecessary

4. Advantages of Scoped Values

#	Advantage	Explanation	Real-world benefit
1	No parameter plumbing	Context doesn't need to be passed through every method	Cleaner APIs
2	Bounded lifetime	Value exists only within its scope	Prevents stale context
3	Immutable by design/use	Intended for one-way transmission	Safer shared context
4	Efficient inheritance	Works efficiently across structured child threads	Excellent with virtual threads
5	Better than ThreadLocal for one-way context	Callee cannot simply replace the caller's binding	Better encapsulation
6	Automatic cleanup	Binding reverts when	No manual remove()

		scope ends	
7	Supports nested scopes	Inner scopes can temporarily rebind a value	Flexible context propagation
8	Thread sharing	Values can be inherited by structured child threads	Useful for concurrent applications
9	Good for request context	Request ID, user context, tracing information	Cleaner service code
10	Good with Structured Concurrency	Child tasks can inherit scoped bindings	Excellent modern concurrency model
11	Virtual-thread friendly	Designed with large numbers of virtual threads in mind	Scalable server applications
12	Reduces accidental mutation	Intended for one-way transmission	Better reasoning
13	Thread-local pitfalls avoided	No indefinite thread-associated value	Fewer lifecycle bugs
14	Fast repeated access	Implementation uses a scoped-value cache	Efficient get() access

The JDK documentation specifically recommends `ScopedValue` over `ThreadLocal` when the goal is one-way transmission of data without method parameters, and notes its efficient inheritance with structured task scopes. ([Oracle Docs](#))

5. Disadvantages of Scoped Values

#	Disadvantage	Explanation	Concern
1	Preview feature in Java 23	Not yet permanent	Production compatibility risk
2	Implicit dependency	Method signatures don't reveal that a scoped value is required	Can make dependencies less obvious
3	Requires careful naming/design	Global-looking keys can become confusing	Maintainability
4	Not suitable for mutable state	Designed primarily for immutable/context data	Wrong abstraction for changing state

5	Not a universal ThreadLocal replacement	Some ThreadLocal use cases need mutable per-thread state	Choose carefully
6	Scope must be understood	Value is available only within its dynamic scope	Can cause NoSuchElementException if unbound
7	Overuse can hide architecture	Too many implicit values make code harder to understand	Context pollution
8	Limited number recommended	The implementation uses a small per-thread cache	Excessive scoped values can hurt performance
9	Migration effort	Existing ThreadLocal-heavy applications may require redesign	Refactoring cost
10	Not ideal for business data	Ordinary values are usually better as parameters	Avoid unnecessary implicit state
11	Preview compilation required	Java 23 requires preview support	Deployment complexity
12	Requires structured thinking	Developers must understand dynamic scope	Learning curve

The Java 23 API documentation notes that applications should minimize the number of scoped values used because the per-thread cache is small; if many values need to be transmitted, grouping them into a record and binding one scoped value is suggested. ([Oracle Docs](#))

6. When should you use Scoped Values?

Situation	Use?	Reason
Request ID	☑ Yes	Cross-cutting request context
Trace/correlation ID	☑ Yes	Needed deep in call chain
Security/user context	☑ Yes, carefully	Context propagation
Tenant ID	☑ Yes	Multi-tenant request context

Locale for current operation	☑ Yes	Cross-cutting contextual data
Transaction/request context	☑ Yes, carefully	Shared contextual metadata
Logging context	☑ Yes	Avoid passing IDs through every layer
Structured-concurrency child tasks	☑ Excellent	Values can be inherited
Virtual-thread applications	☑ Excellent	Efficient context sharing
Immutable configuration for an operation	☑ Yes	Read-only contextual data
Ordinary method argument	☑ No	Use a parameter
Mutable application state	☑ No	Use proper state management
Global application configuration	☑ No	Use configuration/service objects
Long-term storage	☑ No	Use database/cache
Communication between unrelated requests	☑ No	Use explicit messaging/storage

7. When should you NOT use it?

Scenario	Why avoid it?	Better choice
Simple method-to-method data passing	Explicit parameter is clearer	Method parameter
Mutable state	Scoped values aren't intended for this	Proper state object
Application-wide configuration	Scope is unnecessary	Configuration object
Data that must survive request completion	Scope ends	Database/cache

Background job context	May outlive original scope	Explicit job context
Complex dependency injection	Hidden dependencies become difficult	DI framework
Large number of context values	Cache efficiency can suffer	Context record/object
Public API design	Hidden dependency isn't obvious	Explicit parameter
Independent asynchronous tasks	Scoped lifetime may not match	Explicit context propagation
Persistent user/session data	Wrong lifetime	Session/database

8. Real-time Use Case — Request ID

This is one of the strongest use cases.

Suppose:

```

HTTP Request
  |
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Logger

```

Every layer needs the request ID for logging.

Without ScopedValue

```

void controller(String requestId) {
    service(requestId);
}

void service(String requestId) {
    repository(requestId);
}

```

```
void repository(String requestId) {
    log(requestId);
}
```

This creates **parameter plumbing**.

With ScopedValue

```
static final ScopedValue<String> REQUEST_ID =
    ScopedValue.newInstance();

void handleRequest(String requestId) {
    ScopedValue.runWhere(
        REQUEST_ID,
        requestId,
        () -> service()
    );
}

void service() {
    repository();
}

void repository() {
    log();
}

void log() {
    IO.println("Request = " + REQUEST_ID.get());
}
```

Now:

```
handleRequest()
├── REQUEST_ID
│   ├── service()
│   ├── repository()
│   └── log()
```

This is exactly the kind of one-way contextual transmission for which scoped values are intended. ([Oracle Docs](#))

9. Real-time Use Case — Distributed Tracing

Consider a microservice request:

API Gateway
↓
Order Service
↓
Payment Service
↓
Inventory Service

All services/methods need:

```
traceId = "TRACE-9876"
```

Within a Java process, a scoped value can carry the trace context through the call tree:

```
static final ScopedValue<String> TRACE_ID =  
    ScopedValue.newInstance();
```

```
ScopedValue.runWhere(  
    TRACE_ID,  
    "TRACE-9876",  
    () -> processOrder()  
);
```

Deep inside:

```
void processPayment() {  
    IO.println("Trace = " + TRACE_ID.get());  
}
```

This can make logging/tracing code cleaner.

Important: In a distributed system, the trace ID still needs to be propagated across network boundaries explicitly, typically through HTTP/message metadata. ScopedValue does not magically transmit values between JVM processes.

10. Real-time Use Case — Multi-Tenant Application

Suppose:

Request
↓
Tenant A
↓
Controller
↓
Service
↓

Repository

You can bind:

```
static final ScopedValue<String> TENANT_ID =  
    ScopedValue.newInstance();  
  
ScopedValue.runWhere(  
    TENANT_ID,  
    "TENANT-A",  
    () -> processRequest()  
);
```

Then:

```
void repositoryOperation() {  
    String tenant = TENANT_ID.get();  
  
    // Use tenant to select appropriate data  
}
```

This avoids passing:

```
tenantId  
tenantId  
tenantId  
tenantId
```

through every method.

Security caution: Do not treat ScopedValue itself as an authorization mechanism. Authorization should still be enforced explicitly.

11. Real-time Use Case — Locale / Request Preferences

A request may contain:

```
Language = en-IN  
Currency = INR  
TimeZone = Asia/Kolkata
```

Instead of passing these through every method:

```
static final ScopedValue<RequestContext> CONTEXT =  
    ScopedValue.newInstance();  
  
record RequestContext{
```

```
String language,
String currency,
String timeZone
) {}
```

Then:

```
ScopedValue.runWhere(
    CONTEXT,
    new RequestContext("en-IN", "INR", "Asia/Kolkata"),
    () -> generateInvoice()
);
```

Deep inside:

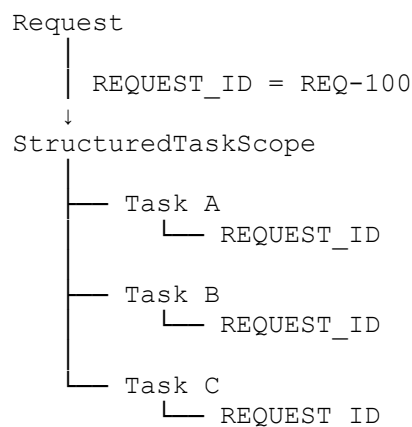
```
void generateInvoice() {
    RequestContext context = CONTEXT.get();

    // Format invoice using request context
}
```

This is especially useful when several contextual values belong together. The Java 23 documentation specifically suggests grouping multiple values into a record when necessary rather than creating many scoped values. ([Oracle Docs](#))

12. Real-time Use Case — Structured Concurrency

Scoped Values become especially interesting when combined with Structured Concurrency.



The Java 23 API specifies that when a StructuredTaskScope is created, scoped-value bindings are captured and inherited by threads started through `fork()`. ([Oracle Docs](#))

Example conceptually:

```
static final ScopedValue<String> REQUEST_ID =  
    ScopedValue.newInstance();
```

```
void handleRequest() throws Exception {
```

```
    ScopedValue.runWhere(  
        REQUEST_ID,  
        "REQ-100",  
        () -> executeConcurrentTasks()  
    );  
}
```

The child tasks can access:

```
REQUEST_ID.get()
```

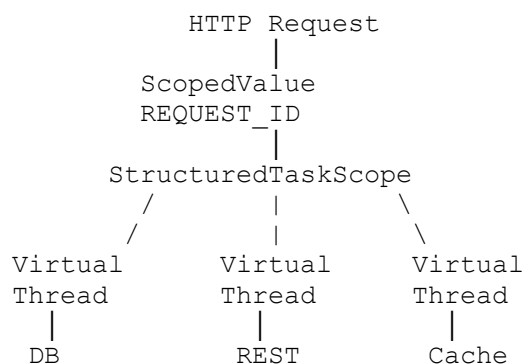
without explicitly passing the request ID to each task.

13. Scoped Values + Virtual Threads

This combination is particularly important in modern Java.

Technology	Responsibility
Virtual Threads	Efficiently execute large numbers of concurrent tasks
Scoped Values	Efficiently propagate immutable contextual data
Structured Concurrency	Manage the lifetime/cancellation of related tasks

Together:



Oracle's Java 23 migration documentation specifically notes that scoped values are preferred over thread-local variables, especially when using large numbers of virtual threads. ([Oracle Docs](#))

14. Scoped Values vs ThreadLocal

This is the **most important comparison**.

Aspect	ThreadLocal	ScopedValue
Introduced	Older Java API	Java 20+ preview
Main purpose	Per-thread state	Scoped contextual data
Mutability	Mutable	Intended for immutable/one-way sharing
Lifetime	Can persist with thread	Bounded by dynamic scope
Cleanup	Often requires remove()	Automatic scope exit
Caller controls value	Not necessarily	Binding controlled by scope
Callee can change value	✓	Not through normal rebinding of caller's binding
Virtual threads	Can work, but context management can be costly	Designed for efficient sharing
Inheritance	More expensive	Designed for efficient structured inheritance
Nested rebinding	Manual pattern	Built in
Best use	Mutable per-thread state	Immutable contextual data
Request context	?	? Excellent
Logging/tracing	?	? Excellent
Mutable transaction state	?	?
Legacy frameworks	?	?

The JDK documentation specifically identifies three issues with ThreadLocal that scoped values address: callees can change the value, the value can persist after the operation unless removed, and inheritance can be expensive. ([Oracle Docs](#))

15. Scoped Values vs Method Parameters

Requirement	Method Parameter	Scoped Value
Explicit dependency	★★★★★	★★
Readability	★★★★★	★★★
Request context	★★★	★★★★★
Deep call-chain context	★★	★★★★★
API clarity	★★★★★	★★★
Cross-cutting context	★★★	★★★★★
Ordinary business data	★★★★★	★
Hidden dependency	None	Possible
Refactoring safety	Excellent	Good but requires context awareness

Rule

If the value is part of the method's business input, use a parameter. If it is contextual information needed by many methods deep in a call chain, consider a Scoped Value.

16. Scoped Values vs Dependency Injection

Aspect	Scoped Value	Dependency Injection
Primary purpose	Context propagation	Object dependency management
Request ID	Excellent	Possible but usually excessive
Service dependency	✗	Excellent
Database repository	✗	Excellent
Configuration	?	Excellent

Logging context	Excellent	Possible
User/request context	Excellent	Possible
Object lifecycle	Scope-based	Container-managed
Framework dependency	No	Often yes
Best for	Context	Application architecture

Scoped Values should **not** replace dependency injection.

17. Alternate features / technologies

Alternative	When to use instead	Comparison
ThreadLocal	Legacy applications or mutable per-thread state	Closest traditional alternative
Method parameters	Ordinary business data	Most explicit and type-safe
Request Context object	Passing multiple related values	Explicit alternative
Dependency Injection	Object/service dependencies	Different purpose
MDC	Logging context	Specialized logging solution
CompletableFuture context propagation	Existing async pipelines	More manual
Reactive Context	Reactive applications	Better for Reactor/WebFlux-style systems
Framework request context	Spring/Jakarta applications	Framework-managed lifecycle
Message metadata	Cross-process context	Required for distributed propagation

18. Which alternative should you choose?

Your requirement	Best choice
-------------------------	--------------------

Simple value between two methods	Method parameter
Request ID through deep synchronous call chain	ScopedValue
Request ID in legacy thread-based application	ThreadLocal / MDC
Mutable thread-specific state	ThreadLocal
Multiple contextual values	ScopedValue<ContextRecord>
Service dependency	Dependency Injection
Logging-only context	MDC / logging framework context
Reactive pipeline context	Reactive Context
Cross-service propagation	HTTP/message metadata
Persistent user information	Database/session/cache

19. Performance considerations

Scoped Values can provide important efficiency benefits, but they are **not simply a general-purpose performance optimization**.

The Java 23 implementation uses a small per-thread cache for scoped-value lookups. Repeated `get()` operations are generally very fast after caching, while using many scoped values cyclically can reduce cache effectiveness. ([Oracle Docs](#))

Situation	Expected suitability
Small number of scoped values	👍 Excellent
Repeated <code>get()</code>	👍 Very good
Virtual-thread applications	👍 Excellent
One request context record	👍 Excellent
Hundreds of independently cycling scoped values	👎 Poor design
Replacing every method parameter	👎 Poor design

Mutable shared state	❓ Wrong abstraction
-----------------------------	---------------------

A good pattern is:

```
record RequestContext(  
    String requestId,  
    String tenantId,  
    String traceId  
) {}
```

```
static final ScopedValue<RequestContext> CONTEXT =  
    ScopedValue.newInstance();
```

rather than:

```
ScopedValue<String> REQUEST_ID;  
ScopedValue<String> TENANT_ID;  
ScopedValue<String> TRACE_ID;  
ScopedValue<String> USER_ID;  
ScopedValue<String> LOCALE;  
ScopedValue<String> ...
```

20. Nested scopes / rebinding

Scoped Values support **rebinding**.

Suppose:

Outer scope
REQUEST_ID = REQ-100

↓

Inner scope
REQUEST_ID = REQ-200

Inside the inner scope:

```
REQUEST_ID.get()
```

returns:

REQ-200

After the inner scope completes:

REQUEST_ID = REQ-100

The outer value is restored automatically. ([Oracle Docs](#))

Conceptually:

```
ScopedValue.runWhere(  
    REQUEST_ID,  
    "REQ-100",  
    () -> {  
  
        // REQ-100  
  
        ScopedValue.runWhere(  
            REQUEST_ID,  
            "REQ-200",  
            () -> {  
                // REQ-200  
            }  
        );  
  
        // REQ-100 again  
    }  
);
```

This is much cleaner than manually saving/restoring thread-local values.

21. Common mistakes

Mistake	Problem	Better approach
Using ScopedValue for every variable	Creates hidden dependencies	Use parameters
Storing mutable objects	Can reintroduce shared-state problems	Prefer immutable records
Creating dozens of scoped values	Can hurt cache efficiency	Group context into a record
Treating it as global state	Scope semantics are different	Keep bindings operation-scoped
Using it for persistence	Value disappears after scope	Database/cache
Assuming it crosses JVMs	It doesn't	Explicit network/message propagation

Using it as an authorization mechanism	Not a security boundary	Explicit authorization
Ignoring unbound state	get() can fail if no binding exists	Establish scope deliberately
Replacing DI with ScopedValue	Different responsibilities	Keep service dependencies explicit

22. Decision table

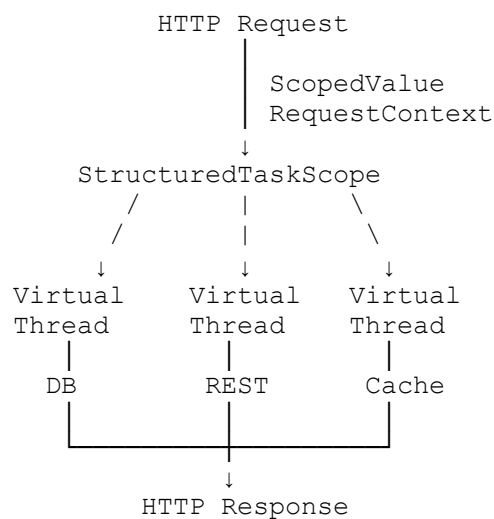
Requirement	Scoped Value?	Recommended approach
Request ID	☑ Yes	ScopedValue
Trace ID	☑ Yes	ScopedValue
Tenant ID	☑ Yes	ScopedValue
User/request context	☑ Yes	ScopedValue + immutable record
Locale/contextual preferences	☑ Yes	ScopedValue
Structured child tasks	☑ Excellent	ScopedValue + Structured Concurrency
Virtual-thread context propagation	☑ Excellent	ScopedValue
Ordinary business parameter	☐	Method parameter
Mutable per-thread state	☐	ThreadLocal
Service dependency	☐	Dependency Injection
Persistent state	☐	Database/cache
Cross-process context	☐	HTTP/message metadata
Reactive stream context	☐	Reactive Context

23. Scoped Values + Structured Concurrency + Virtual Threads

These three Java features complement each other extremely well:

Feature	Problem solved
Virtual Threads	How to efficiently run many concurrent tasks
Structured Concurrency	How to organize/cancel/manage related concurrent tasks
Scoped Values	How to propagate immutable contextual information to those tasks

Together:



This is one of the strongest modern-Java concurrency patterns.

24. Final recommendation

Category	Recommendation
Request context	🔗 Excellent
Tracing/correlation ID	🔗 Excellent
Tenant context	🔗 Excellent
Logging context	🔗 Excellent

Virtual-thread applications	👉 Excellent
Structured Concurrency applications	👉 Excellent
Immutable contextual information	👉 Excellent
Ordinary business data	👉 Use method parameters
Mutable state	👉 Prefer appropriate state management
Global configuration	👉 Don't use
Persistent data	👉 Don't use
Cross-process communication	👉 Don't use directly
Java 23 production adoption	👉 Caution — preview feature
Replacement for all ThreadLocals	👉 No — migrate selectively

Bottom line

Scoped Values are best understood as a safer, bounded-lifetime mechanism for propagating immutable contextual data through a call chain—especially request IDs, trace IDs, tenant context, and request metadata. They are particularly powerful when combined with Virtual Threads and Structured Concurrency. They should not replace ordinary method parameters, dependency injection, persistent storage, or every existing ThreadLocal.

Java 23 officially listed Scoped Values as a **third-preview API**, and Oracle specifically recommends them over ThreadLocal for one-way data transmission and highlights their suitability for applications using large numbers of virtual threads. ([Oracle Docs](#))

Java 23 — Flexible Constructor Bodies (Second Preview)

Aspect	Details
Java Version	23
Year	2024
Feature	Flexible Constructor Bodies
UniqueClassName	FlexibleConstructorBodies

JEP	482
Status	Second Preview
First introduced	Java 22 as a Preview
Main purpose	Allow statements before an explicit constructor invocation such as <code>super(...)</code>
Traditional restriction	<code>super(...)</code> or <code>this(...)</code> must be the first statement
New capability	Certain initialization/validation statements can execute before <code>super(...)</code>
Primary benefit	More flexible constructor initialization
Primary risk	More complicated constructor reasoning if used excessively
Performance impact	Primarily a language expressiveness improvement; no automatic performance gain

1. What problem does Flexible Constructor Bodies solve?

Traditionally, Java requires the explicit constructor invocation to be the first statement:

```
class Employee extends Person {
    Employee(String name) {
        super(name);
    }
}
```

You **cannot** traditionally do:

```
class Employee extends Person {
    Employee(String name) {
        name = name.trim(); // Not allowed before super()
        super(name);
    }
}
```

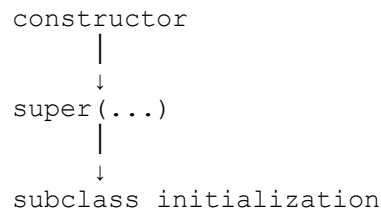
This can be inconvenient when the subclass needs to:

1. Validate constructor arguments.
2. Normalize values.
3. Calculate values needed by `super(...)`.
4. Perform simple preparation before invoking the superclass constructor.

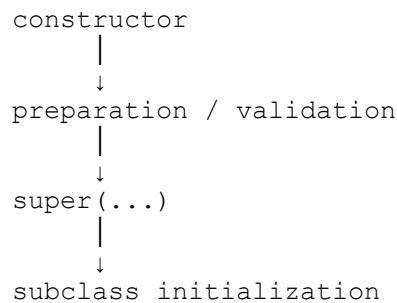
Java 23's Flexible Constructor Bodies allow certain statements to appear before `super(...)` or `this(...)`.

Conceptually:

Traditional constructor



Flexible constructor



2. Basic example

Traditional approach

You might have to put the preparation into a separate static method:

```
class Employee extends Person {

    Employee(String name) {
        super(normalize(name));
    }

    static String normalize(String name) {
        return name.trim();
    }
}
```

With Flexible Constructor Bodies:

```
class Employee extends Person {  
  
    Employee(String name) {  
        name = name.trim();  
        super(name);  
    }  
}
```

The important difference is that the constructor can perform **prologue statements** before the superclass constructor invocation.

3. Important restriction

Flexible Constructor Bodies **do not mean that anything can be executed before super()**.

Java still protects the fundamental object-construction rules.

For example, you cannot use this before the superclass constructor has completed.

Conceptually:

```
class Employee extends Person {  
  
    Employee(String name) {  
        // Allowed:  
        name = name.trim();  
  
        // Then:  
        super(name);  
  
        // After super():  
        this.name = name;  
    }  
}
```

The feature is therefore better described as:

Flexible placement of certain statements before constructor invocation, while preserving Java's object-initialization safety rules.

4. Advantages

#	Advantage	Explanation	Real-world benefit
1	More flexible constructors	Statements can execute before super()/this()	More natural initialization
2	Argument validation	Constructor arguments can be checked before delegation	Fail earlier
3	Input normalization	Values can be trimmed/transformed first	Cleaner constructor calls
4	Better readability	Preparation can stay inside the constructor	Less helper-method clutter
5	Simplifies subclass construction	Values can be prepared before superclass initialization	Cleaner inheritance
6	Less boilerplate	Some static helper methods become unnecessary	Smaller code
7	Useful for calculated arguments	Constructor arguments can be computed first	Easier complex initialization
8	Better encapsulation	Preparation logic can remain close to construction	Improved locality
9	Improves migration/refactoring	Existing constructors can be expressed more naturally	Easier code evolution
10	Preserves initialization safety	Restrictions still prevent illegal use of the object	Safer than arbitrary pre-super execution
11	Supports defensive preparation	Inputs can be checked before parent construction	Better robustness
12	Useful with inheritance-heavy designs	Subclasses can prepare state before delegation	Cleaner subclass constructors

5. Disadvantages

#	Disadvantage	Explanation	Concern
---	--------------	-------------	---------

1	Preview feature in Java 23	Still not final	Compatibility risk
2	More complex constructor rules	Developers must understand what is legal before super()	Learning curve
3	Can reduce readability if abused	Too much logic before super() makes constructors complicated	Maintainability
4	Doesn't remove constructor-order constraints	Java still restricts access to object state before superclass construction	Developers must understand restrictions
5	Inheritance complexity remains	Constructors in deep hierarchies can still be difficult	Architectural issue
6	No inherent performance benefit	More flexible syntax doesn't necessarily make execution faster	Don't use for performance alone
7	Potential style inconsistency	Teams may disagree about how much logic belongs before super()	Code standards needed
8	Can encourage large constructors	Developers may put too much preparation logic into constructors	Harder testing
9	Existing alternatives already work	Helper methods can solve many cases	Benefit isn't universal
10	Preview syntax requires preview support	Java 23 requires appropriate compiler/runtime settings	Deployment complication
11	Constructor dependencies can become hidden	Complex preparation may obscure what the parent needs	Harder reasoning
12	Doesn't solve poor inheritance design	It improves constructor syntax, not class architecture	Composition may still be better

6. When should you use it?

Situation	Use?	Reason
Normalize constructor input	☒ Yes	Very natural use

Validate constructor arguments	☑ Yes	Validation can happen before parent construction
Calculate argument for super()	☑ Yes	Avoids awkward helper methods
Prepare immutable constructor data	☑ Yes	Cleaner initialization
Subclass requires preprocessing	☑ Yes	Directly addresses the problem
Small constructor prologue	☑ Yes	Improves readability
Temporary calculations	☑ Yes	Keeps logic local
Simple constructors	☑ Not necessary	Traditional constructor is already clear
Large initialization logic	☑ No	Move logic elsewhere
Complex business processing	☑ No	Constructors shouldn't become workflows
Side effects	☑ Usually no	Makes construction harder to reason about
Production code requiring Java 23 preview	☑ Caution	Preview status

7. When should you NOT use it?

Scenario	Why avoid it?	Better approach
Simple super(...) call	No problem to solve	Normal constructor
Complex business logic	Constructor becomes difficult to maintain	Factory/service
Database calls	Constructors shouldn't perform expensive I/O	Service/factory
Network calls	Side effects during construction	Service layer

Long calculations	Makes object construction expensive	Factory/precomputation
Large validation workflow	Too much constructor responsibility	Validator/factory
Deep inheritance hierarchy	Constructor complexity compounds	Prefer composition
Public library requiring stable syntax	Preview feature risk	Standard constructor syntax
Logic shared across many constructors	Duplication risk	Helper/factory
Object creation requires multiple decisions	Constructor isn't ideal	Factory Method/Builder

8. Real-time Use Case — Input Normalization

Suppose a superclass requires a normalized username:

```
class User {
    User(String username) {
        // superclass initialization
    }
}
```

Traditional solution:

```
class Admin extends User {

    Admin(String username) {
        super(normalize(username));
    }

    static String normalize(String username) {
        return username.trim().toLowerCase();
    }
}
```

Flexible Constructor Body:

```
class Admin extends User {

    Admin(String username) {
```

```
        username = username.trim().toLowerCase();
    }
    super(username);
}
```

Benefit

The preparation is immediately visible:

```
raw input
↓
trim
↓
lowercase
↓
super(username)
```

9. Real-time Use Case — Validation

Suppose:

```
class Account {
    Account(String accountNumber) {
        // initialize account
    }
}
```

A subclass may need to validate the input:

```
class PremiumAccount extends Account {

    PremiumAccount(String accountNumber) {

        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException(
                "Account number is required"
            );
        }

        super(accountNumber);
    }
}
```

The intent is:

Validate input

↓
If valid
↓
Initialize parent

rather than:

Initialize parent
↓
Discover invalid input

This can be particularly useful when invalid arguments would make superclass initialization inappropriate.

10. Real-time Use Case — Calculating a Parent Constructor Argument

Suppose the parent expects a calculated value:

```
class Employee {  
    Employee(String displayName) {  
        // initialize  
    }  
}
```

Subclass:

```
class Developer extends Employee {  
  
    Developer(String firstName, String lastName) {  
  
        String displayName =  
            firstName.trim() + " " + lastName.trim();  
  
        super(displayName);  
    }  
}
```

The constructor becomes conceptually:

firstName + lastName
↓
displayName
↓
super(displayName)

Without flexible constructor bodies, developers may need to move the calculation into another method:

```
super(createDisplayName(firstName, lastName));
```

The latter is still perfectly valid, but the new feature can make simple preparation more readable.

11. Real-time Use Case — Defensive Constructor Processing

Imagine:

```
class Order {  
    Order(String orderId) {  
        // initialize  
    }  
}
```

Subclass:

```
class InternationalOrder extends Order {  
  
    InternationalOrder(String orderId) {  
  
        orderId = orderId.trim();  
  
        if (orderId.length() < 5) {  
            throw new IllegalArgumentException(  
                "Invalid order ID"  
            );  
        }  
  
        super(orderId);  
    }  
}
```

This keeps simple defensive preparation close to construction.

12. Real-time Use Case — Unit Conversion

Suppose a superclass expects meters:

```
class Distance {  
    Distance(double meters) {  
        // initialize  
    }  
}
```

Subclass receives centimeters:

```

class CentimeterDistance extends Distance {

    CentimeterDistance(double centimeters) {

        double meters = centimeters / 100.0;

        super(meters);
    }
}

```

The constructor performs:

```

centimeters
  ↓
meters
  ↓
super(meters)

```

This is a good example because the preparation is small and deterministic.

13. Real-time Use Case — Security/Domain Validation

Consider a domain object:

```

class Customer {
    Customer(String customerId) {
        // parent initialization
    }
}

```

Subclass:

```

class CorporateCustomer extends Customer {

    CorporateCustomer(String customerId) {

        customerId = customerId.trim();

        if (!customerId.startsWith("CORP-")) {
            throw new IllegalArgumentException(
                "Invalid corporate customer ID"
            );
        }

        super(customerId);
    }
}

```

The constructor ensures that the superclass receives an acceptable value.

However: authentication/authorization should not be implemented merely as constructor logic. This is only an example of domain validation.

14. Traditional Constructor vs Flexible Constructor Body

Aspect	Traditional constructor	Flexible Constructor Body
super() first	Required	No longer strictly first
Preparation before super()	Restricted	Supported within rules
Input validation before parent construction	Awkward	Easier
Argument transformation	Helper method often needed	Can be local
Readability	Good for simple constructors	Better for some preprocessing
Constructor rules	Simpler	More sophisticated
Preview in Java 23	No	Yes
Production compatibility	High	Preview concern
Performance	Normal	No inherent improvement

15. Flexible Constructor Bodies vs Static Helper Method

Approach	Example	Advantages	Disadvantages
Helper method	<code>super(normalize(name))</code>	Explicit and reusable	Extra method
Flexible body	<code>name = name.trim(); super(name);</code>	Local and readable	Preview feature
Factory	<code>create(name)</code>	Handles complex creation	More architecture
Builder	<code>builder().name(...)</code>	Complex objects	Overkill for simple

			cases
--	--	--	-------

Rule of thumb

Use a flexible constructor body when the preparation is:

- short,
- deterministic,
- directly related to construction,
- easy to understand.

Use a helper/factory when the preparation becomes substantial.

16. Flexible Constructor Bodies vs Factory Method

Requirement	Flexible Constructor	Factory Method
Simple normalization	?	?
Simple validation	?	?
Calculate super() argument	? Excellent	?
Complex creation logic	?	? Excellent
Multiple implementation choices	?	?
Database lookup	?	?
Network operation	?	?
Caching objects	?	?
Readability for tiny preparation	?	?
Java 23 preview dependency	Yes	No

17. Flexible Constructor Bodies vs Builder

Situation	Flexible Constructor	Builder
1-3 simple values	?	Usually unnecessary
Many optional fields	?	?
Complex object creation	?	?
Immutable object construction	?	?
Validation of several fields	?	?
Inheritance-heavy construction	?	?
Fluent API required	?	?
Simple subclass preprocessing	? Excellent	? Overkill

18. Alternate features

There is no exact one-to-one replacement because Flexible Constructor Bodies changes **constructor syntax and initialization ordering**.

Alternative	Best use case	Relationship
Traditional constructor + <code>super(...)</code>	Standard production Java	Closest direct alternative
Static helper method	Prepare arguments	Most common workaround
Factory Method	Complex creation	Better when construction has decisions
Builder Pattern	Many optional parameters	Better for complex objects
Static factory method	Validation + controlled creation	Excellent alternative
Composition	Avoid complex inheritance	Architectural alternative

Records	Simple immutable data carriers	Alternative when inheritance isn't needed
Constructor delegation with this(...)	Shared constructor initialization	Useful for overloaded constructors

19. Important alternative — Static Factory Method

Instead of:

```
class PremiumCustomer extends Customer {

    PremiumCustomer(String id) {

        id = id.trim();

        if (id.isBlank()) {
            throw new IllegalArgumentException();
        }

        super(id);
    }
}
```

You can use:

```
class PremiumCustomer extends Customer {

    private PremiumCustomer(String id) {
        super(id);
    }

    static PremiumCustomer create(String id) {

        id = id.trim();

        if (id.isBlank()) {
            throw new IllegalArgumentException();
        }

        return new PremiumCustomer(id);
    }
}
```

This can be preferable when creation requires substantial validation or business logic.

20. Important alternative — Composition

Sometimes Flexible Constructor Bodies make an inheritance problem look easier when the real solution is **not using inheritance**.

Instead of:

```
BaseOrder
  ↑
InternationalOrder
  ↑
PremiumInternationalOrder
```

consider:

```
Order
├── Customer
├── ShippingPolicy
├── PricingPolicy
└── ValidationPolicy
```

Situation	Recommendation
Genuine "is-a" relationship	Inheritance
Behavior varies independently	Composition
Deep inheritance hierarchy	Prefer composition
Constructor chain becoming complicated	Consider composition
Many subclass-specific initialization rules	Consider composition

21. Performance impact

Flexible Constructor Bodies are **not primarily a performance feature**.

Area	Expected impact
CPU performance	No inherent improvement
Memory usage	No significant inherent improvement
Startup time	No meaningful direct benefit

Object creation	Essentially determined by actual constructor logic
Source code size	Can become smaller
Developer productivity	Potentially improved
Readability	Potentially improved
Runtime optimization	JVM still optimizes resulting bytecode normally

For example:

```
String normalized = name.trim();
super(normalized);
```

doesn't become faster merely because the statements are now syntactically permitted before `super()`.

The benefit is primarily:

language expressiveness
+
constructor readability
+
less boilerplate

22. Common mistakes

Mistake	Problem	Better approach
Putting too much logic before super()	Constructor becomes difficult to read	Keep prologue short
Performing database operations	Constructor gets side effects	Factory/service
Calling network services	Slow/unreliable construction	Service layer
Doing complex business rules	Wrong responsibility	Domain/service layer
Using mutable global state	Hard-to-test construction	Explicit dependencies
Assuming everything is allowed before super()	Java still has strict restrictions	Follow constructor-body rules

Using it merely because it's new	Preview feature may not justify migration	Use when it solves a real problem
Ignoring inheritance design	Constructor syntax won't fix bad hierarchy	Consider composition

23. When to choose which approach?

Requirement	Best choice
Simple constructor	Traditional constructor
Normalize one or two arguments	Flexible Constructor Body
Validate simple arguments	Flexible Constructor Body
Calculate super() arguments	Flexible Constructor Body
Complex validation	Factory
Database-backed creation	Factory/service
Multiple creation strategies	Factory
Many optional parameters	Builder
Shared initialization across constructors	this(...) delegation
Deep inheritance hierarchy	Consider composition
Simple immutable data	Record
Stable Java 21/17 production code	Traditional constructor

24. Final recommendation

Category	Recommendation
Simple argument preprocessing	🔗 Excellent
Simple validation	🔗 Excellent
Calculating super() arguments	🔗 Excellent

Small subclass initialization	🔗 Good
Teaching modern Java	🔗 Good
Complex business logic	🔗 Avoid
I/O/network/database work	🔗 Avoid
Very large constructors	🔗 Avoid
Deep inheritance hierarchy	🔗 Reconsider architecture
Long-lived production code on Java 23	🔗 Preview caution
Performance optimization	🔗 Not the reason to use it
Stable production alternative	Traditional constructors + helper/factory methods

Bottom line

Flexible Constructor Bodies (Second Preview) make Java constructors more expressive by allowing limited preparation and validation before `super(...)` or `this(...)`. They are most useful when a subclass needs to normalize, validate, or calculate constructor arguments before delegating to another constructor. Keep the pre-constructor logic small; for complex creation logic, prefer factories, builders, or composition.